

Université Cadi Ayyad

École Nationale des Sciences Appliquées de Marrakech

Filière : Réseaux, Systèmes et Services
Programmables

Mise en place d'un Honeypot pour la détection et l'analyse d'attaques réseau

Projet de fin de module

Module : Sécurité des Infrastructures IT

Réalisées par :

- Berrami Malak
- El Moutanabbih Imane
- Ettouri Maroua
- Rsaim Marwa

Encadrée par :

Mme Wissam ABBASS

Année universitaire : 2024–2025

ENSA Marrakech

Table des matières

Introduction générale	1
1 Déploiement et Architecture du Honeypot	2
1.1 Présentation générale	2
1.2 Choix et installation du honeypot	2
1.2.1 Choix du honeypot	2
1.2.2 Installation de l'environnement	3
1.2.3 Étapes d'installation et lancement de Cowrie	3
1.3 Mise en place de la machine virtuelle et snapshots	4
1.3.1 Configuration de la machine virtuelle	4
1.3.2 Snapshots	4
1.4 Confinement réseau et sécurisation du honeypot	5
1.4.1 Configuration réseau du honeypot	5
1.4.2 Vérification des interfaces	5
1.4.3 Mise en place du pare-feu <code>iptables</code>	6
1.4.4 Vérification du pare-feu	7
1.4.5 Tests d'isolation et de confinement	8
1.4.6 Scan externe et test SSH	9
1.4.7 Analyse du trafic	10
1.5 Architecture globale du système	11
1.6 Conclusion du Chapitre 2	11
2 Analyse et Exploitation des Logs du Honeypot	12
2.1 Introduction	12
2.2 Architecture et Fonctionnement de la Stack ELK	12
2.2.1 Architecture générale	12
2.3 Installation et Configuration de la Stack ELK	13
2.3.1 Installation d'Elasticsearch, Logstash et Kibana	13
2.3.2 Rôle détaillé des composants	13
2.4 Collecte et Vérification des Logs Cowrie	13
2.4.1 Localisation des logs	13
2.4.2 Observation des logs en temps réel	14
2.5 Configuration du Pipeline Logstash	15
2.5.1 Création du fichier pipeline	15
2.5.2 Explication du pipeline	16
2.6 Intégration dans Kibana	17
2.6.1 Accès à l'interface Kibana	17

2.6.2	Création de l'index pattern	17
2.7	Analyse et Visualisation avec Kibana Lens	18
2.7.1	Volume des attaques par jour	18
2.7.2	Top IP attaquantes	19
2.7.3	Usernames les plus testés	19
2.8	Synthèse	19
2.9	Analyse des attaques capturées par le honeypot	20
2.9.1	Attaque 1 — Connexion SSH simple	20
2.9.2	Attaque 2 — Connexion SSH avec plusieurs tentatives	20
2.9.3	Attaque 3 — Scan Nmap (-Pn)	21
2.9.4	Attaque 4 — Scan complet (Top 1000 ports)	22
2.9.5	Attaque 5 — Tentatives SSH avec différents utilisateurs	23
2.9.6	Attaque 6 — Connexion SSH avec clé privée invalide	23
2.9.7	Attaque 7 — Connexion directe via Netcat	24
2.9.8	Attaque 8 — Flood de connexions TCP vers le port 2222	24
2.10	Synthèse générale des attaques observées	25
2.11	Tableau comparatif	27
2.12	Schéma de cycle d'une attaque	28
2.13	Analyse des visualisations Kibana	28
2.13.1	Graphique 1 — Activité dans le temps	28
2.13.2	Graphique 2 — Répartition des adresses IP attaquantes	29
2.13.3	Graphique 3 — Répartition des types d'événements Cowrie	30
2.13.4	Graphique 4 — Volume des événements sur plusieurs jours	31
2.13.5	Graphique 5 — Répartition des noms d'utilisateur testés	32
2.13.6	Graphique 6 - Distribution des sessions ouvertes dans Cowrie	32
2.13.7	Dashboard Global	34
2.13.8	Interprétation générale des graphiques	34
2.14	Conclusion du Chapitre 3	34
	Conclusion du Chapitre 3	34
3	Difficultés techniques rencontrées et méthodes de résolution	35
3.1	Blocage initial de Kibana sur le port 5601	35
3.2	Tentative de contournement via Filebeat	35
3.3	Retour à Logstash et résolution définitive	36
3.4	Conclusion	37
	Conclusion Générale	38

Table des figures

1.1	Exemple de snapshot de la VM Ubuntu avec Cowrie installé	4
1.2	Résultat de la commande <code>ip route</code>	5
1.3	Résultat de la commande <code>ip addr</code>	6
1.4	Exécution du script de configuration du pare-feu	6
1.5	Vérification de succès du pare-feu	8
1.6	Vérification des règles actives du pare-feu	8
1.7	Vérification d'écoute Cowrie	8
1.8	Résultats des tests de connectivité et d'isolation	9
1.9	Résultat du scan Nmap depuis la machine hôte	9
1.10	Journal de Cowrie affichant une tentative de connexion SSH capturée .	10
1.11	Analyse du trafic capturé sous Wireshark	10
1.12	Architecture globale du système Honeypot Cowrie et pipeline ELK . . .	11
2.1	Installation de la Stack ELK depuis les dépôts officiels	13
2.2	Installation de Logstash responsable du pipeline d'ingestion	13
2.3	Installation de Kibana, l'interface de visualisation	13
2.4	Installation du moteur d'indexation Elasticsearch	13
2.5	Observation en temps réel des journaux Cowrie avec tail	14
2.6	Analyse des événements Cowrie via journalctl	15
2.7	Configuration du pipeline Logstash pour Cowrie	16
2.8	Interface d'accueil de Kibana après installation	17
2.9	Détection des index dans Elasticsearch	17
2.10	Création de l'index pattern <code>cowrie-*</code>	18
2.11	Évolution quotidienne du volume d'attaques enregistrées	18
2.12	Les adresses IP les plus actives dans les tentatives d'intrusion	19
2.13	Noms d'utilisateurs les plus ciblés dans les attaques bruteforce	19
2.14	Tentative de connexion SSH simple détectée par Cowrie (<code>cowrie.login.failed</code>)	20
2.15	Multiples tentatives SSH capturées par Cowrie	21
2.16	Scan Nmap exécuté avec l'option <code>-Pn</code>	22
2.17	Scan Nmap des 1000 ports les plus utilisés	22
2.18	Tentatives SSH multiples avec différents usernames	23
2.19	Tentative de connexion SSH avec clé invalide	24
2.20	Connexion brute via Netcat	24
2.21	Flood de connexions TCP successives vers le honeypot	25
2.22	Cycle d'une attaque capturée par Cowrie et analyse via la stack ELK .	28
2.23	Évolution temporelle des événements Cowrie	28
2.24	Origine des tentatives de connexion	29
2.25	Types d'événements générés lors des attaques	30

2.26	Volume quotidien des événements Cowrie	31
2.27	Usernames utilisés lors des attaques SSH	32
2.28	Distribution des sessions ouvertes détectées dans Cowrie	33
2.29	Tableau de bord complet regroupant les différentes visualisations	34
3.1	Test du port 5601 : absence de réponse de Kibana	35
3.2	Filebeat correctement installé sur le système	36
3.3	Filebeat : le service démarre puis s'arrête immédiatement	36
3.4	Synthèse des difficultés techniques rencontrées et de leur impact sur le pipeline ELK	37

Liste des tableaux

1.1	Interfaces réseau configurées pour la VM honeypot	5
2.1	Tableau comparatif des différentes attaques capturées par Cowrie . . .	27

Introduction générale

La sécurité des systèmes d'information est devenue un enjeu majeur dans un monde où les cyberattaques ne cessent d'évoluer. Face à ces menaces, les organisations doivent adopter des solutions capables de détecter, analyser et prévenir les attaques potentielles. C'est dans ce cadre que s'inscrit notre projet, qui consiste à **mettre en place un honeypot** permettant d'observer les comportements malveillants et d'améliorer la compréhension des techniques d'intrusion.

L'objectif principal est de créer un environnement contrôlé simulant des vulnérabilités afin d'attirer les attaquants, tout en enregistrant leurs activités pour analyse. Ce travail nous a permis de renforcer nos connaissances en sécurité réseau, en analyse de trafic et en détection d'intrusions.

Ce rapport est structuré comme suit :

- Le premier chapitre détaille la **mise en place du honeypot** et les différentes configurations réalisées.
- Le deuxième chapitre est consacré à **l'analyse des attaques détectées** et à l'interprétation des résultats obtenus.
- Le troisième chapitre est consacré à **Difficultés techniques rencontrées et méthodes de résolution**.
- Enfin, nous terminerons par une **conclusion générale** résumant les apports du projet et les perspectives futures.

Chapitre 1

Déploiement et Architecture du Honeypot

1.1 Présentation générale

Dans un contexte où les infrastructures informatiques sont de plus en plus exposées aux cyberattaques, le recours à une solution de type honeypot devient pertinent. Un honeypot est un système ou une ressource réseau délibérément configuré pour paraître vulnérable, afin d’attirer les attaquants, d’enregistrer leurs actions, et de fournir des informations sur leurs techniques.

Son rôle principal est donc double :

- Détourner l’attention des attaquants loin des actifs critiques.
- Analyser les comportements des attaquants pour améliorer la posture de sécurité

On utilise un honeypot car il offre un moyen proactif de défense : il ne repose pas uniquement sur la protection passive (pare-feux, antivirus), mais il permet de « voir l’ennemi » en action, de collecter des logs, d’analyser des vecteurs d’attaque émergents, et d’ajuster les mesures de sécurité en conséquence.

1.2 Choix et installation du honeypot

1.2.1 Choix du honeypot

Après étude des différentes solutions disponibles, le choix s’est porté sur **Cowrie**. Les raisons sont les suivantes :

- Cowrie est un honeypot orienté SSH/Telnet (et d’autres services) qui permet de capturer les tentatives d’intrusion, les commandes malveillantes et les données laissées par l’attaquant.
- Il est relativement léger à déployer, bien documenté, et adapté à une phase expérimentale dans un environnement étudiant.
- Il offre une bonne granularité d’observation tout en restant sécurisé dans un environnement isolé.

1.2.2 Installation de l'environnement

L'installation a été faite sur une machine virtuelle Ubuntu (version standard). Avant le lancement de l'installation, plusieurs préparations ont été réalisées :

- Création de la VM avec deux adaptateurs réseau : un NAT pour mise à jour / internet, et un réseau interne pour isoler le honeypot.
- Configuration de l'environnement virtuel (RAM, stockage, snapshots initiaux).
- Mise à jour du système Ubuntu et installation des prérequis (Python, Git, dépendances de Cowrie).

1.2.3 Étapes d'installation et lancement de Cowrie

Les commandes exécutées et leur rôle sont les suivants :

1. Mise à jour du système :

```
1 sudo apt update
2 sudo apt upgrade -y
```

Cette commande met à jour la liste des paquets et installe les mises à jour disponibles.

2. Installation des dépendances :

```
1 sudo apt install python3 python3-venv python3-pip git
```

Installe Python 3, le module venv pour créer un environnement virtuel, pip pour gérer les packages, et git pour cloner le dépôt.

3. Clonage du dépôt Cowrie :

```
1 git clone https://github.com/cowrie/cowrie.git cowrie-latest
2 cd cowrie-latest
```

Télécharge la dernière version de Cowrie depuis GitHub et se place dans le dossier.

4. Création de l'environnement virtuel Python :

```
1 python3 -m venv cowrie-env
2 source cowrie-env/bin/activate
3 pip install --upgrade pip setuptools wheel
4 pip install -r requirements.txt
5 pip install -e .
```

Crée un environnement isolé pour Cowrie, installe les dépendances et active le package.

5. Lancement de Cowrie :

```
1 cowrie-env/bin/cowrie start
2 cowrie-env/bin/cowrie status
```

Démarre Cowrie et vérifie l'état du honeypot.

1.3 Mise en place de la machine virtuelle et snapshots

1.3.1 Configuration de la machine virtuelle

Voici les caractéristiques et étapes utilisées :

- Nom de la VM : **ubuntupro**
- Système d'exploitation : Ubuntu
- Mémoire RAM : 4 Go
- Processeurs : 2
- Réseau :
 - Adaptateur 1 : NAT → pour accès internet / mises à jour.
 - Adaptateur 2 : Réseau interne (intnet ou host-only) → pour isolement du honeypot.

1.3.2 Snapshots

Avant tout changement majeur, des snapshots ont été créés pour permettre un retour arrière en cas de problème :

1. Snapshot après installation de Ubuntu (avant toute configuration Cowrie).
2. Snapshot après configuration réseau et installation des dépendances.
3. Snapshot après installation de Cowrie et tests initiaux (honeypot-ready).

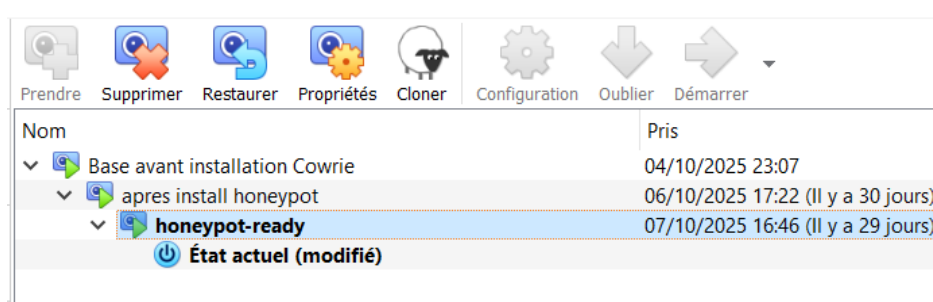


FIGURE 1.1 – Exemple de snapshot de la VM Ubuntu avec Cowrie installé

1.4 Confinement réseau et sécurisation du honeypot

1.4.1 Configuration réseau du honeypot

La machine hébergeant le honeypot Cowrie a été configurée avec plusieurs interfaces réseau afin de séparer les flux et limiter les risques d'interférences entre les environnements. Cette segmentation garantit que le trafic capturé par le honeypot reste confiné à un espace de test isolé.

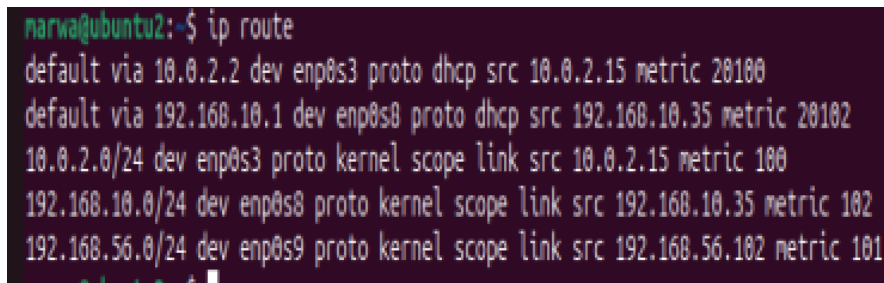
Interface	Type	Fonction	Adresse IP (exemple)
enp0s3	NAT	Connexion Internet	Dynamique
enp0s8	Interne	Réseau isolé de test	192.168.10.10
enp0s9	Host-Only	Accès supervision	192.168.56.102

TABLE 1.1 – Interfaces réseau configurées pour la VM honeypot

1.4.2 Vérification des interfaces

Avant toute configuration du pare-feu, il est essentiel d'identifier les interfaces actives et leurs adresses IP. Les commandes suivantes permettent de visualiser les interfaces et la table de routage du système.

```
1 ip addr
2 ip route
```



```
marwa@ubuntu2:~$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp src 10.0.2.15 metric 20100
default via 192.168.10.1 dev enp0s8 proto dhcp src 192.168.10.35 metric 20102
10.0.2.0/24 dev enp0s3 proto kernel scope link src 10.0.2.15 metric 100
192.168.10.0/24 dev enp0s8 proto kernel scope link src 192.168.10.35 metric 102
192.168.56.0/24 dev enp0s9 proto kernel scope link src 192.168.56.102 metric 101
marwa@ubuntu2:~$
```

FIGURE 1.2 – Résultat de la commande `ip route`

```

marwa@ubuntu2:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:3e:7d:af brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 86274sec preferred_lft 86274sec
    inet6 fd00::415f:40c1:c7b1:33e2/64 scope global temporary dynamic
        valid_lft 86276sec preferred_lft 14276sec
    inet6 fd00::a00:27ff:fe3e:7daf/64 scope global dynamic mngtppaddr
        valid_lft 86276sec preferred_lft 14276sec
    inet6 fe80::a00:27ff:fe3e:7daf/64 scope link
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:e3:0b:d0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.35/24 brd 192.168.10.255 scope global dynamic noprefixroute enp0s8
        valid_lft 7075sec preferred_lft 7075sec
    inet6 fe80::a5b0:c001:6968:0609/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: enp0s9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b3:10:49 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.102/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s9
        valid_lft 474sec preferred_lft 474sec
    inet6 fe80::500d:af05:36dd:2b6/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

FIGURE 1.3 – Résultat de la commande `ip addr`

1.4.3 Mise en place du pare-feu iptables

Afin d'assurer la sécurité et l'isolation du honeypot, un pare-feu basé sur `iptables` a été mis en place. Ce pare-feu empêche toute communication non désirée entre la machine honeypot et les autres réseaux, tout en autorisant uniquement les flux nécessaires au fonctionnement de Cowrie.

```

marwa@ubuntu2:~$ sudo nano ~/honeypot-firewall.sh
[sudo] Mot de passe de marwa :

```

FIGURE 1.4 – Exécution du script de configuration du pare-feu

Le script suivant automatise l'ensemble de la configuration et de la journalisation des règles :

```

1 #!/bin/bash
2 set -e
3 iptables-save > ~/iptables-backup-$(date +%F_%T).rules

```

```

4
5 iptables -F
6 iptables -X
7 iptables -t nat -F
8 iptables -t mangle -F
9
10 # Autorise cowrie ssh_honeypot
11 iptables -A INPUT -i enp0s9 -p tcp --dport 2222 -j ACCEPT
12 iptables -A OUTPUT -o enp0s9 -p tcp --sport 2222 -j ACCEPT
13 iptables -t nat -A PREROUTING -i enp0s9 -p tcp --dport 22 -j REDIRECT
    ↪ --to-port 2222
14
15 iptables -P INPUT DROP
16 iptables -P OUTPUT DROP
17 iptables -P FORWARD DROP
18
19 iptables -A INPUT -i lo -j ACCEPT
20 iptables -A OUTPUT -o lo -j ACCEPT
21
22 iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
23 iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED,RELATED -j
    ↪ ACCEPT
24
25 iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
26 iptables -A INPUT -p icmp --icmp-type echo-reply -j ACCEPT
27 iptables -A OUTPUT -p icmp -j LOG --log-prefix "HONEYPOT_ICMP_OUT:_"
28 iptables -A OUTPUT -p icmp -j DROP
29
30 iptables -A OUTPUT -o enp0s8 -j LOG --log-prefix "HONEYPOT_OUT_INT:_"
31 iptables -A OUTPUT -o enp0s8 -j DROP
32 iptables -A OUTPUT -d 192.168.10.0/24 -j LOG --log-prefix "
    ↪ HONEYPOT_TO_INT:_"
33 iptables -A OUTPUT -d 192.168.10.0/24 -j DROP
34 iptables -A FORWARD -i enp0s8 -j DROP
35 iptables -A FORWARD -o enp0s8 -j DROP
36
37 # JOURNALISATION FINALE
38 iptables -A INPUT -j LOG --log-prefix "HONEYPOT_INPUT_DROP:_"
39 iptables -A OUTPUT -j LOG --log-prefix "HONEYPOT_OUTPUT_DROP:_"
40
41 mkdir -p /etc/iptables
42 iptables-save > /etc/iptables/rules.v4
43 echo "_Pare-feu configuré avec succès!_"

```

L'état des tables iptables est exporté via iptables-save. Les règles essentielles sont : politique par défaut DROP, accept loopback, accept ESTABLISHED/RELATED, accept TCP d'entrée sur 2222 pour enp0s3/enp0s9, DROP/LOG pour OUTPUT vers enp0s8 et 192.168.10.0/24.

1.4.4 Vérification du pare-feu

Après création du script, il est rendu exécutable puis lancé afin d'appliquer les règles. Les commandes ci-dessous permettent ensuite de vérifier le bon fonctionnement du pare-feu :

```

1 chmod +x honeypot-firewall.sh

```

```
2 sudo ./honeypot-firewall.sh
3 sudo iptables -L -v -n --line-numbers
```

```
marwa@ubuntu2:~$ sudo ./honeypot-firewall.sh
Pare-feu Hoenypot activé avec succès.
```

FIGURE 1.5 – Vérification de succès du pare-feu

```
marwa@ubuntu2:~$ sudo iptables -L -v -n --line-numbers
Chain INPUT (policy DROP 170 packets, 9375 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1    72 6000 ACCEPT     0  --  lo     *       0.0.0.0/0             0.0.0.0/0
2    17 3096 ACCEPT     0  --  *      *       0.0.0.0/0             0.0.0.0/0           ctstate RELATED,ESTABLISHED
3     2  104 ACCEPT     6  --  enp0s9 *       0.0.0.0/0             0.0.0.0/0           tcp dpt:2222
4     0   0 ACCEPT     1  --  *      *       0.0.0.0/0             0.0.0.0/0           icmptype 8
5     0   0 ACCEPT     1  --  *      *       0.0.0.0/0             0.0.0.0/0           icmptype 0
6     0   0 ACCEPT     6  --  enp0s9 *       0.0.0.0/0             0.0.0.0/0           tcp dpt:2222
7    170 9375 LOG      0  --  *      *       0.0.0.0/0             0.0.0.0/0           LOG flags 0 level 4 prefix "HONEYPOT_INPUT_DROP:"

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1     0   0 DROP      0  --  enp0s8 *       0.0.0.0/0             0.0.0.0/0
2     0   0 DROP      0  --  *      *       0.0.0.0/0             0.0.0.0/0

Chain OUTPUT (policy DROP 732 packets, 53952 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1    72 6000 ACCEPT     0  --  *      lo     0.0.0.0/0             0.0.0.0/0
2    14 1992 ACCEPT     0  --  *      *       0.0.0.0/0             0.0.0.0/0           ctstate RELATED,ESTABLISHED
3     0   0 ACCEPT     6  --  *      enp0s9 0.0.0.0/0             0.0.0.0/0           tcp spt:2222
4     0   0 LOG      1  --  *      *       0.0.0.0/0             0.0.0.0/0           LOG flags 0 level 4 prefix "HONEYPOT_ICMP_OUT:"
5     0   0 DROP      1  --  *      *       0.0.0.0/0             0.0.0.0/0
6     0   0 ACCEPT     6  --  *      enp0s9 0.0.0.0/0             0.0.0.0/0           tcp spt:2222
7   691 49894 LOG      0  --  *      enp0s8 0.0.0.0/0             0.0.0.0/0           LOG flags 0 level 4 prefix "HONEYPOT_OUT_INT:"
8   691 49894 DROP      0  --  *      enp0s8 0.0.0.0/0             0.0.0.0/0
9     0   0 LOG      0  --  *      *       0.0.0.0/0             192.168.10.0/24       LOG flags 0 level 4 prefix "HONEYPOT_TO_INT:"
10    0   0 DROP      0  --  *      *       0.0.0.0/0             192.168.10.0/24
11   731 53892 LOG      0  --  *      *       0.0.0.0/0             0.0.0.0/0           LOG flags 0 level 4 prefix "HONEYPOT_OUTPUT_DROP:"
marwa@ubuntu2:~$
```

FIGURE 1.6 – Vérification des règles actives du pare-feu

L'analyse des règles affichées confirme que le pare-feu fonctionne correctement, avec des politiques par défaut en DROP et des exceptions uniquement pour le service Cowrie et le trafic local.

1.4.5 Tests d'isolation et de confinement

Une fois le pare-feu actif, plusieurs tests ont été effectués afin de vérifier que le honeypot est bien isolé du réseau interne et d'Internet.

Vérification du port Cowrie :

```
1 sudo ss -tunap | grep :2222
```

```
(cowrie-env) marwa@ubuntu2:~/cowrie-latest$ sudo ss -tunlp | grep 2222
tcp LISTEN 0      50      0.0.0.0:2222      0.0.0.0:*    users:(("twistd",pid=2845,fd=11))
(cowrie-env) marwa@ubuntu2:~/cowrie-latest$
```

FIGURE 1.7 – Vérification d'écoute Cowrie

Ce résultat prouve que Cowrie écoute correctement sur le port 2222 tout en restant confiné.

Ensuite, plusieurs pings et connexions ont été testés :

```

1 ping 8.8.8.8
2 ping 192.168.10.1
3 nc -zv 192.168.10.1 22

```

```

marwa@ubuntu2:~$ ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2080ms

marwa@ubuntu2:~$ ping -c 3 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data.

--- 192.168.10.1 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2029ms

```

FIGURE 1.8 – Résultats des tests de connectivité et d'isolation

Les échecs des pings vers 8.8.8.8 et 192.168.10.1 démontrent que la machine honeypot n'a ni accès à Internet ni au réseau interne, ce qui valide la réussite du confinement.

1.4.6 Scan externe et test SSH

Pour s'assurer du bon fonctionnement du honeypot et de sa visibilité simulée depuis l'extérieur, un scan réseau et une tentative de connexion SSH ont été réalisés depuis la machine hôte.

```

1 nmap -sS -p 22 192.168.56.102
2 ssh test@192.168.56.102
3 sudo tail -f /home/cowrie/var/log/cowrie/cowrie.log

```

```

(kali@kali)-[~]
$ nmap -p 22 192.168.56.102
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-11-07 11:02 EST
Nmap scan report for 192.168.56.102
Host is up (0.0025s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 08:00:27:B3:18:49 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 13.26 seconds

```

FIGURE 1.9 – Résultat du scan Nmap depuis la machine hôte


```

(cowrie-env) marwa@ubuntu2:~/cowrie-latest$ tail -f ~/cowrie-latest/var/log/cowrie/cowrie.log
2025-11-07T16:15:45.777483Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] NEW KEYS
2025-11-07T16:15:45.779322Z [cowrie.ssh.transport.HoneyPotSSHTransport#debug] starting service b'ssh-userauth'
2025-11-07T16:15:45.783364Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'marwa' trying auth b'none'
2025-11-07T16:15:49.786533Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'marwa' trying auth b'password'
2025-11-07T16:15:49.787911Z [HoneyPotSSHTransport,0,192.168.56.101] Could not read etc/userdb.txt, default database activated
2025-11-07T16:15:49.789279Z [HoneyPotSSHTransport,0,192.168.56.101] login attempt [b'marwa'/b'123'] failed
2025-11-07T16:15:50.870895Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'marwa' failed auth b'password'
2025-11-07T16:15:50.874323Z [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2025-11-07T16:16:22.038928Z [cowrie.ssh.transport.HoneyPotSSHTransport#info] connection lost
2025-11-07T16:16:22.040045Z [HoneyPotSSHTransport,0,192.168.56.101] Connection lost after 36.3 seconds

```

FIGURE 1.10 – Journal de Cowrie affichant une tentative de connexion SSH capturée

Les résultats du scan montrent que le port 22 semble ouvert (redirigé vers le port 2222 du honeypot), et les logs de Cowrie confirment la capture d’une tentative de connexion SSH. Cela prouve que le honeypot remplit bien son rôle de leurre tout en restant protégé.

1.4.7 Analyse du trafic

Pour garantir que le confinement est complet, des captures réseau ont été réalisées à l’aide de `tcpdump` sur les interfaces principales :

```

1 sudo tcpdump -i enp0s9 -w /tmp/honeypot_enp0s9.pcap
2 sudo tcpdump -i enp0s3 -w /tmp/honeypot_enp0s3.pcap
3 sudo tcpdump -i enp0s8 -w /tmp/honeypot_enp0s8.pcap

```

```

marwa@ubuntu2:~$ sudo tcpdump -i enp0s9 -w /tmp/honeypot_enp0s9.pcap
tcpdump: listening on enp0s9, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C0 packets captured
0 packets received by filter
0 packets dropped by kernel
marwa@ubuntu2:~$ sudo tcpdump -i enp0s3 -w /tmp/honeypot_enp0s3.pcap
tcpdump: listening on enp0s3, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C0 packets captured
0 packets received by filter
0 packets dropped by kernel
marwa@ubuntu2:~$ sudo tcpdump -i enp0s8 -w /tmp/honeypot_enp0s8.pcap
tcpdump: listening on enp0s8, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C1 packet captured
2 packets received by filter
0 packets dropped by kernel

```

FIGURE 1.11 – Analyse du trafic capturé sous Wireshark

Les analyses réalisées confirment qu’aucun trafic sortant n’a été observé sur l’interface Internet, prouvant ainsi que le honeypot est parfaitement isolé et que la configuration du pare-feu est efficace.

1.5 Architecture globale du système

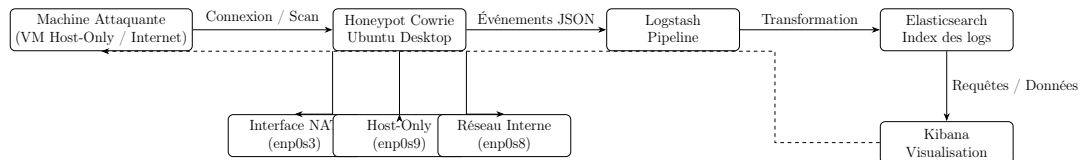


FIGURE 1.12 – Architecture globale du système Honeypot Cowrie et pipeline ELK

1.6 Conclusion du Chapitre 2

Ce chapitre a détaillé la mise en place technique du honeypot Cowrie, depuis le choix de la solution jusqu’à sa configuration complète dans un environnement virtualisé. L’installation de la machine Ubuntu, la configuration réseau, les snapshots et le déploiement du pare-feu ont permis d’obtenir un système isolé, contrôlé et sécurisé, capable de capturer les tentatives d’intrusion sans représenter un risque pour le réseau réel.

L’architecture conçue garantit un confinement strict du honeypot, tout en assurant sa visibilité pour les attaquants. Les différents tests effectués (port scanning, ping, SSH, tcpdump) ont validé la robustesse des mesures de sécurité et le bon fonctionnement du honeypot. Ce travail constitue la base essentielle permettant l’analyse approfondie des attaques, qui sera traitée dans le chapitre suivant.

Chapitre 2

Analyse et Exploitation des Logs du Honeypot

2.1 Introduction

Ce chapitre décrit l'installation de la stack **ELK** (Elasticsearch, Logstash, Kibana) ainsi que l'exploitation des journaux générés par le honeypot Cowrie. Ce travail a été réalisé par **El Moutanabbih Imane**. L'objectif principal est de centraliser les logs, les transformer, et produire des visualisations permettant d'analyser les comportements d'attaquants réels.

L'intégration d'un honeypot avec une plateforme d'analyse telle que ELK constitue une approche moderne et proactive en cybersécurité, car elle permet d'observer des attaques authentiques et d'en extraire des tendances exploitables.

2.2 Architecture et Fonctionnement de la Stack ELK

La stack ELK est constituée de trois composants complémentaires :

- **Elasticsearch** : moteur d'indexation et de recherche.
- **Logstash** : pipeline permettant d'ingérer et transformer les données.
- **Kibana** : interface web de visualisation et d'analyse.

2.2.1 Architecture générale

La communication entre les composants suit le pipeline suivant :

Cowrie → Logstash → Elasticsearch → Kibana

Cette architecture permet :

- une collecte automatisée des événements,
- une transformation standardisée des logs,
- une indexation optimisée pour les recherches,
- une visualisation claire des comportements d'attaque.

2.3 Installation et Configuration de la Stack ELK

2.3.1 Installation d'Elasticsearch, Logstash et Kibana

Les trois composants ont été installés via APT depuis les dépôts officiels Elastic.

```
imanelm@imanelm-VirtualBox:~$ curl -fsSL https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
imanelm@imanelm-VirtualBox:~$ echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elastic-8.x.list
```

FIGURE 2.1 – Installation de la Stack ELK depuis les dépôts officiels

```
imanelm@imanelm-VirtualBox:~$ sudo apt install logstash -y \
> sudo systemctl enable logstash \
> sudo systemctl start logstash \
> sudo systemctl status logstash
```

FIGURE 2.2 – Installation de Logstash responsable du pipeline d'ingestion

```
imanelm@imanelm-VirtualBox:~$ sudo apt install kibana -y \
> sudo systemctl enable kibana \
> sudo systemctl start kibana \
> sudo systemctl status kibana
```

FIGURE 2.3 – Installation de Kibana, l'interface de visualisation

```
imanelm@imanelm-VirtualBox:~$ sudo apt install elasticsearch \
> sudo systemctl enable elasticsearch \
> sudo systemctl start elasticsearch \
> sudo systemctl status elasticsearch
```

FIGURE 2.4 – Installation du moteur d'indexation Elasticsearch

2.3.2 Rôle détaillé des composants

- **Elasticsearch** stocke l'intégralité des événements du honeypot et offre des capacités avancées de recherche et filtrage.
- **Logstash** lit les logs bruts au format JSON, extrait les champs pertinents, reformate les dates et envoie un document propre vers Elasticsearch.
- **Kibana** permet d'explorer les données avec des tableaux de bord, graphes dynamiques et visualisations adaptées à l'analyse des comportements malveillants.

2.4 Collecte et Vérification des Logs Cowrie

2.4.1 Localisation des logs

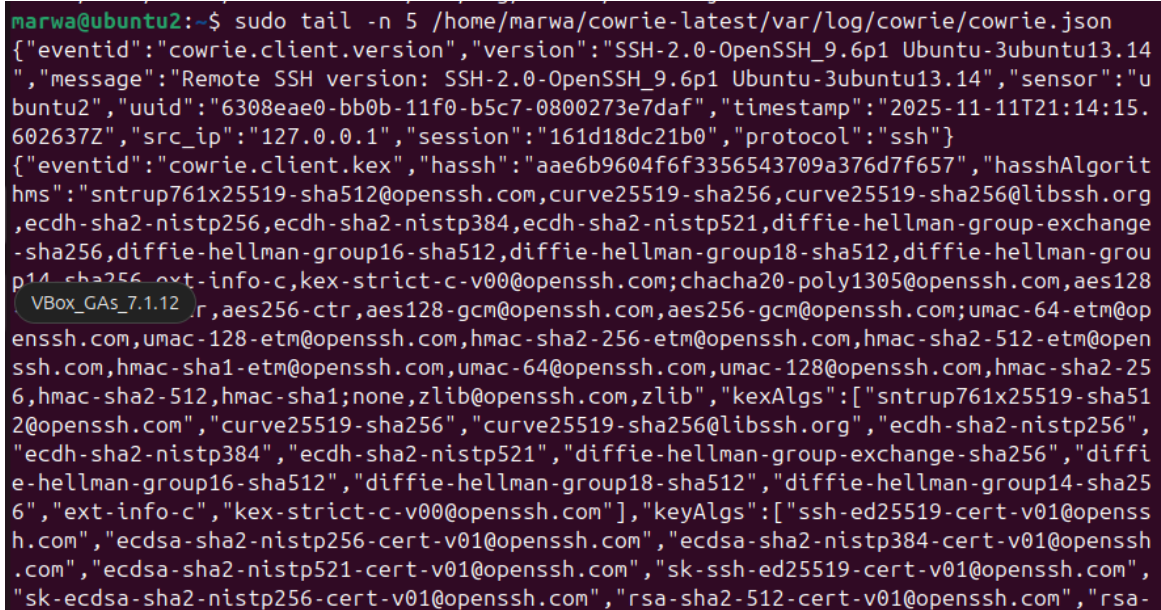
Les journaux générés par Cowrie sont enregistrés dans :

```
1 /home/cowrie/cowrie/var/log/cowrie.json
```

2.4.2 Observation des logs en temps réel

Via tail :

```
1 tail -f /home/cowrie/cowrie/var/log/cowrie.json
```



```
marwa@ubuntu2:~$ sudo tail -n 5 /home/marwa/cowrie-latest/var/log/cowrie/cowrie.json
{"eventid":"cowrie.client.version","version":"SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.14",
,"message":"Remote SSH version: SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.14","sensor":"u
buntu2","uuid":"6308eae0-bb0b-11f0-b5c7-0800273e7daf","timestamp":"2025-11-11T21:14:15.
602637Z","src_ip":"127.0.0.1","session":"161d18dc21b0","protocol":"ssh"}
{"eventid":"cowrie.client.kex","hassh":"aae6b9604f6f3356543709a376d7f657","hasshAlgorit
hms":"sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-sha256@libssh.org
,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521,diffie-hellman-group-exchange
-sha256,diffie-hellman-group16-sha512,diffie-hellman-group18-sha512,diffie-hellman-grou
p14-sha256,ext-info-c,kex-strict-c-v00@openssh.com;chacha20-poly1305@openssh.com,aes128
VBox_GAs_7.1.12r,aes256-ctr,aes128-gcm@openssh.com,aes256-gcm@openssh.com;umac-64-etm@op
enssh.com,umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@open
ssh.com,hmac-sha1-etm@openssh.com,umac-64@openssh.com,umac-128@openssh.com,hmac-sha2-25
6,hmac-sha2-512,hmac-sha1;none,zlib@openssh.com,zlib","kexAlgs":["sntrup761x25519-sha51
2@openssh.com","curve25519-sha256","curve25519-sha256@libssh.org","ecdh-sha2-nistp256",
"ecdh-sha2-nistp384","ecdh-sha2-nistp521","diffie-hellman-group-exchange-sha256","diffi
e-hellman-group16-sha512","diffie-hellman-group18-sha512","diffie-hellman-group14-sha25
6","ext-info-c","kex-strict-c-v00@openssh.com"],"keyAlgs":["ssh-ed25519-cert-v01@openss
h.com","ecdsa-sha2-nistp256-cert-v01@openssh.com","ecdsa-sha2-nistp384-cert-v01@openssh
.com","ecdsa-sha2-nistp521-cert-v01@openssh.com","sk-ssh-ed25519-cert-v01@openssh.com",
"sk-ecdsa-sha2-nistp256-cert-v01@openssh.com","rsa-sha2-512-cert-v01@openssh.com","rsa-
```

FIGURE 2.5 – Observation en temps réel des journaux Cowrie avec tail

Via journalctl :

```
1 journalctl -u cowrie -f
```

```

marwa@ubuntu2:~$ sudo tail -n 5 /home/marwa/cowrie-latest/var/log/cowrie/cowrie.json
{"eventid":"cowrie.client.version","version":"SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.14",
,"message":"Remote SSH version: SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.14","sensor":"u
buntu2","uuid":"6308eae0-bb0b-11f0-b5c7-0800273e7daf","timestamp":"2025-11-11T21:14:15.
602637Z","src_ip":"127.0.0.1","session":"161d18dc21b0","protocol":"ssh"}
{"eventid":"cowrie.client.kex","hassh":"aae6b9604f6f3356543709a376d7f657","hasshAlgorit
hms":["sntrup761x25519-sha512@openssh.com,curve25519-sha256,curve25519-sha256@libssh.org
,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521,diffie-hellman-group-exchange
-sha256,diffie-hellman-group16-sha512,diffie-hellman-group18-sha512,diffie-hellman-grou
p14-sha256,ext-info-c,kex-strict-c-v00@openssh.com;chacha20-poly1305@openssh.com,aes128
VBox_GAs_7.1.12r,aes256-ctr,aes128-gcm@openssh.com,aes256-gcm@openssh.com;umac-64-etm@op
enssh.com,umac-128-etm@openssh.com,hmac-sha2-256-etm@openssh.com,hmac-sha2-512-etm@open
ssh.com,hmac-sha1-etm@openssh.com,umac-64@openssh.com,umac-128@openssh.com,hmac-sha2-25
6,hmac-sha2-512,hmac-sha1;none,zlib@openssh.com,zlib","kexAlgs":["sntrup761x25519-sha51
2@openssh.com","curve25519-sha256","curve25519-sha256@libssh.org","ecdh-sha2-nistp256",
"ecdh-sha2-nistp384","ecdh-sha2-nistp521","diffie-hellman-group-exchange-sha256","diffi
e-hellman-group16-sha512","diffie-hellman-group18-sha512","diffie-hellman-group14-sha25
6","ext-info-c","kex-strict-c-v00@openssh.com"],"keyAlgs":["ssh-ed25519-cert-v01@openss
h.com","ecdsa-sha2-nistp256-cert-v01@openssh.com","ecdsa-sha2-nistp384-cert-v01@openssh
.com","ecdsa-sha2-nistp521-cert-v01@openssh.com","sk-ssh-ed25519-cert-v01@openssh.com",
"sk-ecdsa-sha2-nistp256-cert-v01@openssh.com","rsa-sha2-512-cert-v01@openssh.com","rsa-

```

FIGURE 2.6 – Analyse des événements Cowrie via journalctl

2.5 Configuration du Pipeline Logstash

Le pipeline de Logstash convertit les logs Cowrie en un format exploitable.

2.5.1 Création du fichier pipeline

```

1 /etc/logstash/conf.d/logstash.conf

```

```

GNU nano 7.2                               /etc/logstash/conf.d/c
input {
  file {
    path => "/home/.../cowrie/var/log/cowrie.json"
    codec => json
    start_position => "beginning"
  }
}

filter {
  date {
    match => ["timestamp", "ISO8601"]
    target => "@timestamp"
  }
}

output {
  elasticsearch {
    hosts => ["http://localhost:9200"]
    index => "cowrie-%{+YYYY.MM.dd}"
  }
  stdout { codec => rubydebug }
}

```

FIGURE 2.7 – Configuration du pipeline Logstash pour Cowrie

2.5.2 Explication du pipeline

- **Input** : lecture du fichier JSON généré par Cowrie.
- **Filter** : extraction du timestamp, conversion en format ISO8601.
- **Output** : envoi des événements dans l'index `cowrie-*` de Elasticsearch.

2.6 Intégration dans Kibana

2.6.1 Accès à l'interface Kibana

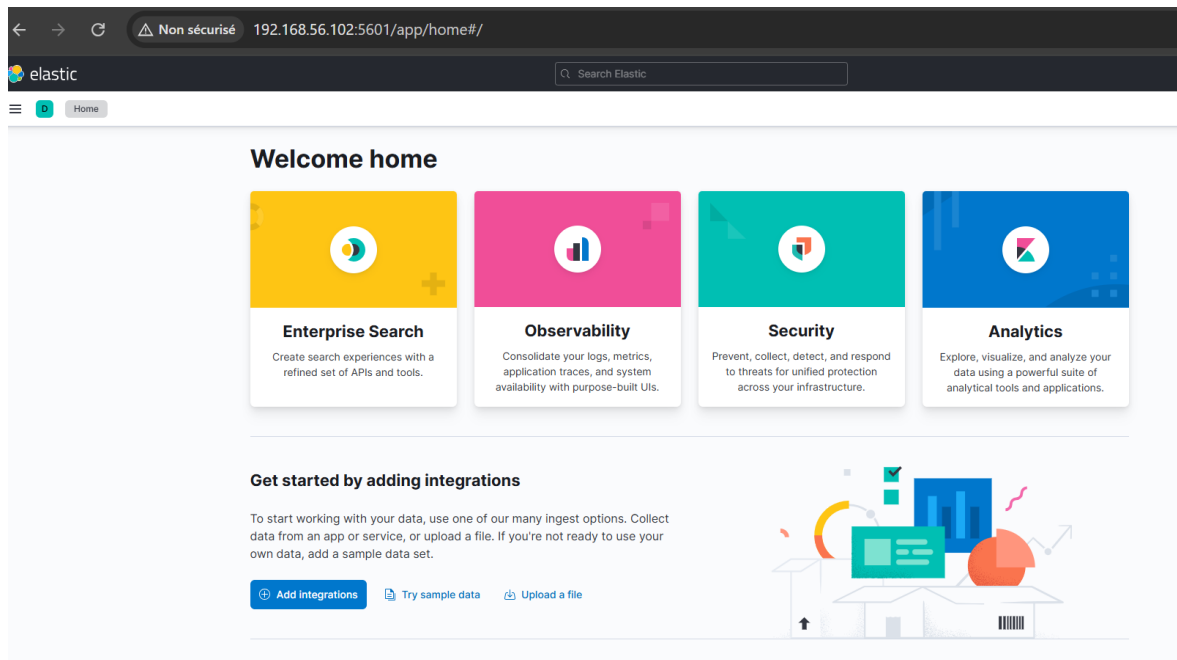


FIGURE 2.8 – Interface d'accueil de Kibana après installation

2.6.2 Création de l'index pattern

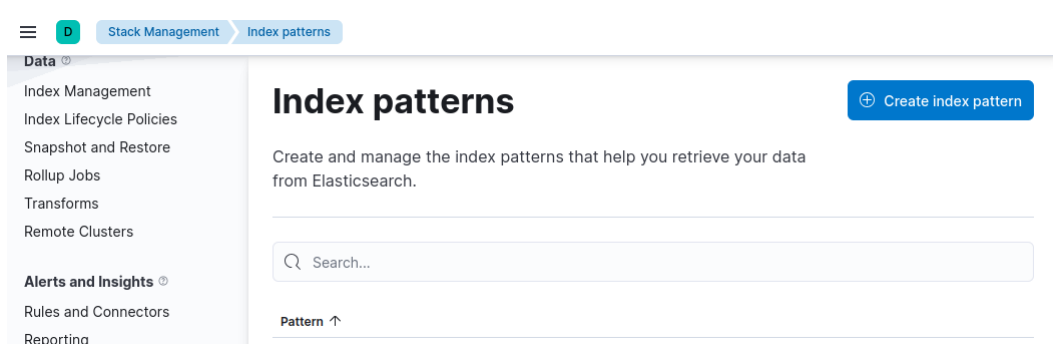


FIGURE 2.9 – Détection des index dans Elasticsearch

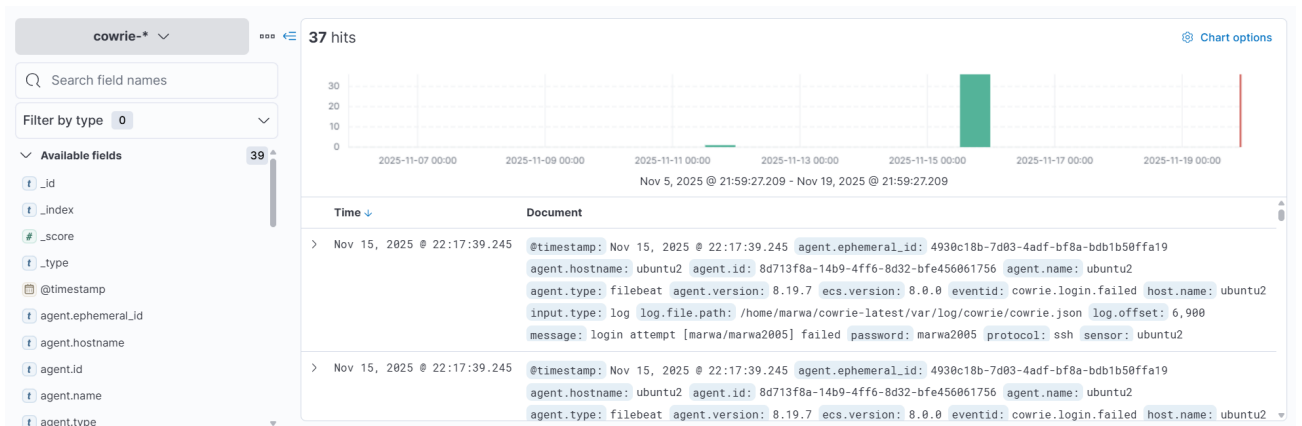


FIGURE 2.10 – Création de l'index pattern `cowrie-*`

2.7 Analyse et Visualisation avec Kibana Lens

2.7.1 Volume des attaques par jour

Objectif : analyser la fréquence quotidienne des tentatives d'intrusion.

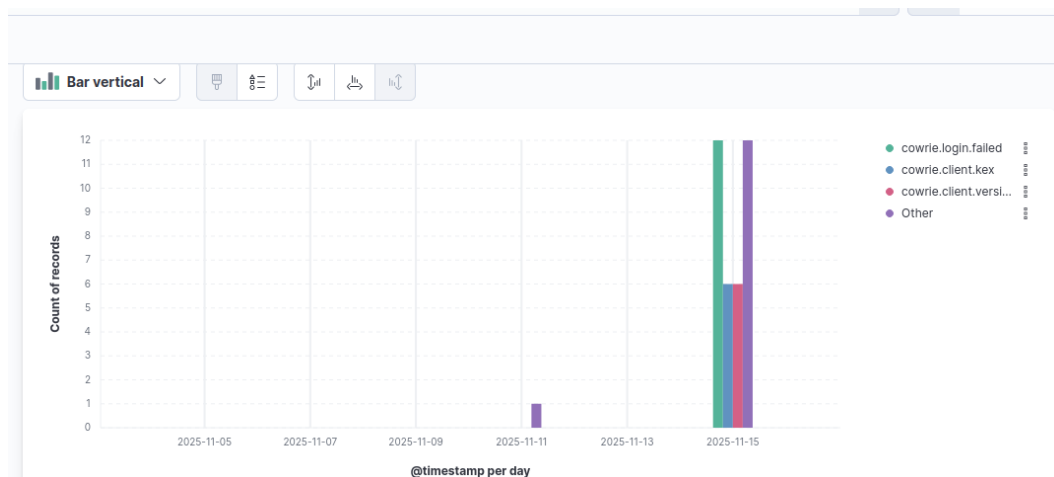


FIGURE 2.11 – Évolution quotidienne du volume d'attaques enregistrées

Analyse : On observe un flux continu de tentatives SSH, avec des pics récurrents. Cela confirme que les attaques automatisées sont constantes et ciblent toutes les machines accessibles sur Internet.

2.7.2 Top IP attaquantes

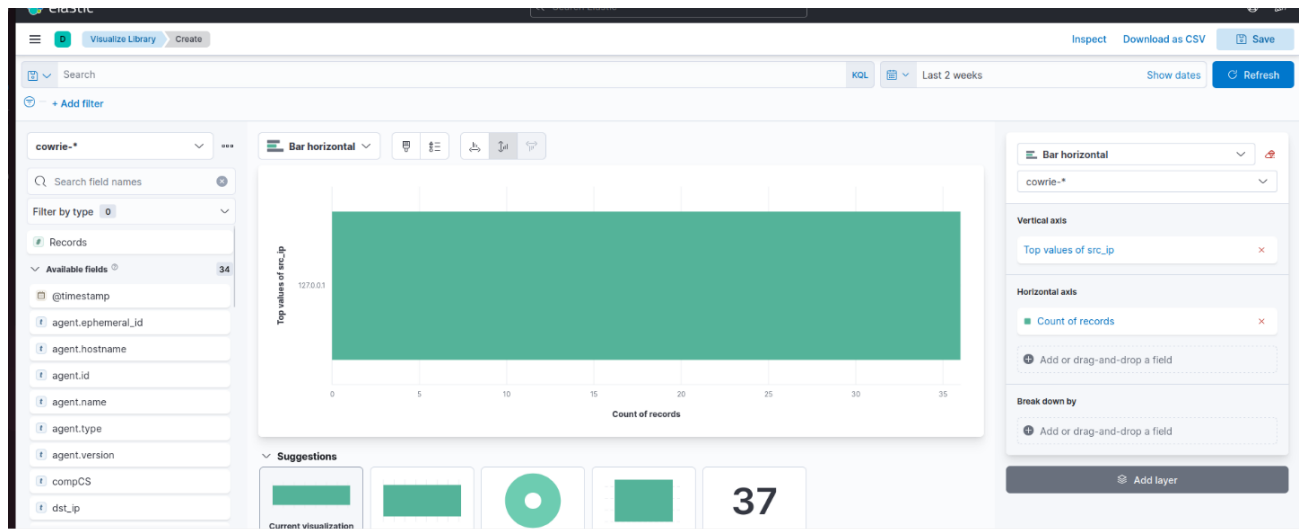


FIGURE 2.12 – Les adresses IP les plus actives dans les tentatives d'intrusion

Analyse : Certaines IP reviennent fréquemment, indiquant l'utilisation probable de botnets ou de serveurs compromis.

2.7.3 Usernames les plus testés

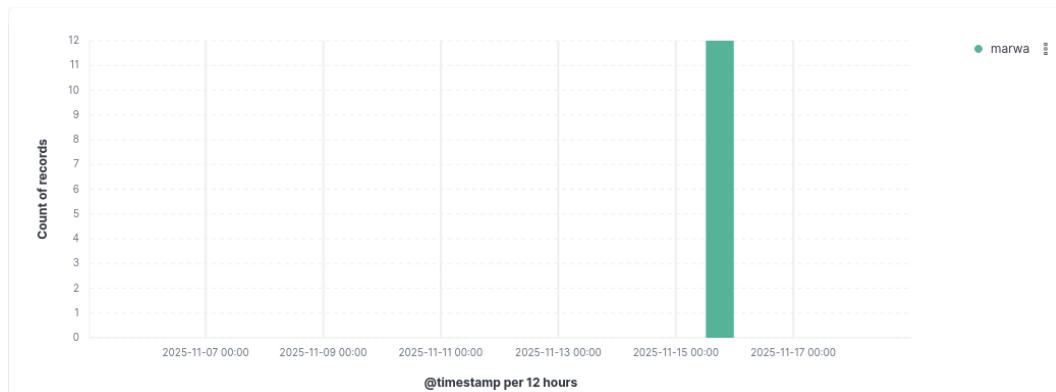


FIGURE 2.13 – Noms d'utilisateurs les plus ciblés dans les attaques bruteforce

Analyse : Les attaquants testent principalement des identifiants par défaut tels que root, admin, ou test. Cela confirme des tentatives automatisées de brute-force.

2.8 Synthèse

La mise en place de la stack ELK associée au honeypot Cowrie a permis une analyse approfondie des attaques. Grâce aux visualisations, il est possible d'identifier les IP les plus agressives, les usernames ciblés, ainsi que la fréquence des attaques.

L'ensemble démontre la pertinence d'un honeypot couplé à une solution d'analyse, offrant une vision claire des menaces et permettant une amélioration continue de la posture de sécurité du système.

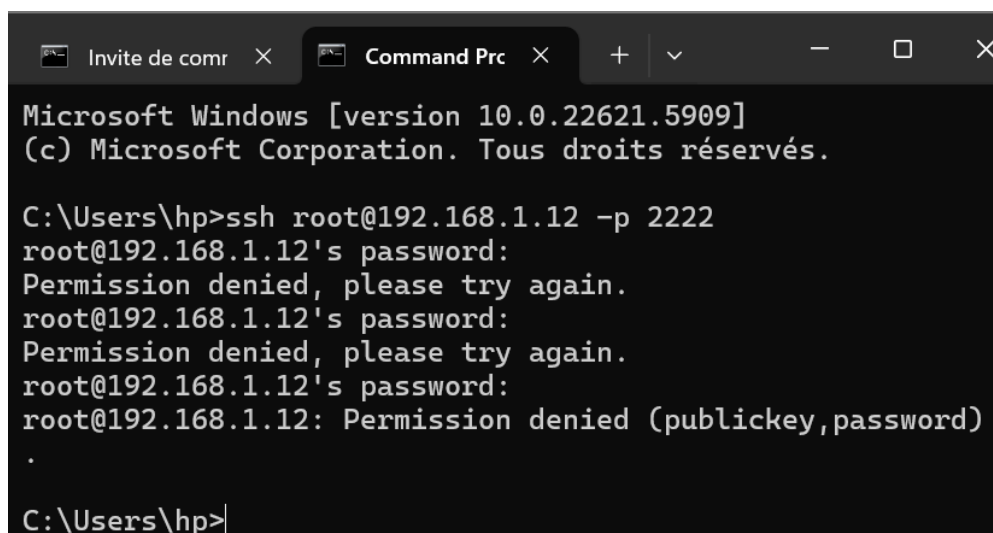
2.9 Analyse des attaques capturées par le honeypot

Cette section présente les différentes attaques réalisées dans le cadre des tests du honeypot Cowrie. Chaque attaque a été observée, documentée et interprétée afin de comprendre le comportement du honeypot et la manière dont il réagit face à diverses formes d'interactions.

2.9.1 Attaque 1 — Connexion SSH simple

La première attaque consiste en une tentative de connexion SSH avec un nom d'utilisateur inventé. Cowrie détecte immédiatement cette tentative et enregistre l'événement suivant :

— `cowrie.login.failed` : échec d'authentification dû à un identifiant incorrect. L'adresse IP source provient de la machine locale utilisée pour les tests.



```
Microsoft Windows [version 10.0.22621.5909]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\hp>ssh root@192.168.1.12 -p 2222
root@192.168.1.12's password:
Permission denied, please try again.
root@192.168.1.12's password:
Permission denied, please try again.
root@192.168.1.12's password:
root@192.168.1.12: Permission denied (publickey,password)
.
```

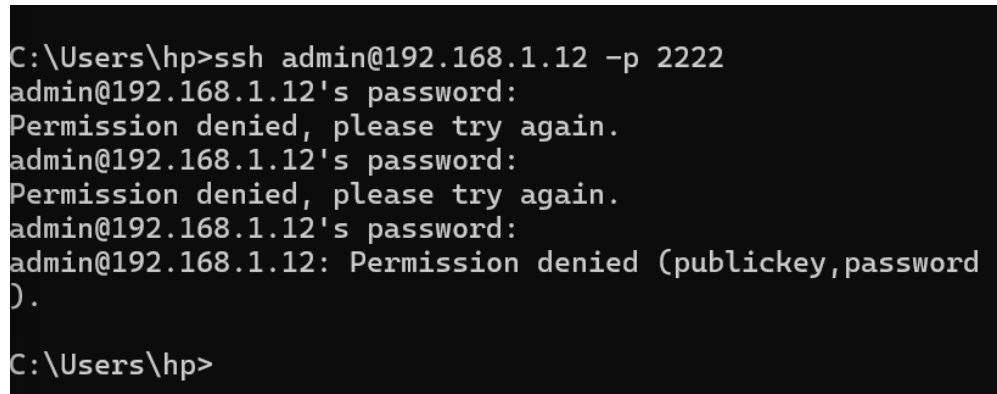
FIGURE 2.14 – Tentative de connexion SSH simple détectée par Cowrie (`cowrie.login.failed`)

Interprétation : Ce type d'événement représente une attaque simple ou un test d'accès non autorisé. Le honeypot réagit correctement en refusant la tentative et en journalisant toutes les informations nécessaires : IP source, username utilisé, timestamp, etc. Cela montre que Cowrie est fonctionnel et capable de capturer des interactions même minimalistes.

2.9.2 Attaque 2 — Connexion SSH avec plusieurs tentatives

Dans ce scénario, plusieurs tentatives successives de mot de passe ont été effectuées sur un même compte SSH. Cowrie enregistre :

- Plusieurs `cowrie.login.failed`
- Des événements `cowrie.session` dès qu'une session est initiée



```
C:\Users\hp>ssh admin@192.168.1.12 -p 2222
admin@192.168.1.12's password:
Permission denied, please try again.
admin@192.168.1.12's password:
Permission denied, please try again.
admin@192.168.1.12's password:
admin@192.168.1.12: Permission denied (publickey,password
).
C:\Users\hp>
```

FIGURE 2.15 – Multiples tentatives SSH capturées par Cowrie

Interprétation : Ce comportement est typique d'une attaque par *brute-force*, où l'attaquant tente de deviner un mot de passe valide. Cowrie enregistre chaque tentative individuellement, ce qui permet d'analyser :

- le nombre d'essais,
- les usernames ciblés,
- le rythme des attaques,
- l'adresse IP source.

Un tel niveau de détail est essentiel pour comprendre les techniques automatisées utilisées par les bots.

2.9.3 Attaque 3 — Scan Nmap (-Pn)

Un scan Nmap avec l'option `-Pn` a été effectué dans le but de détecter les services ouverts sans effectuer de ping préalable. Le scan a tenté une connexion TCP vers le port 2222 (port interne de Cowrie). Cependant, aucune détection n'a été enregistrée par Cowrie : aucun `cowrie.session.connect` n'a été généré.

```
C:\Users\hp>"C:\Program Files (x86)\Nmap\nmap.exe" -Pn -s
T -sV 192.168.1.12 -p 2222
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-20 23:
59 Maroc (heure d♦♦t♦♦)
Nmap scan report for 192.168.1.12
Host is up (0.0010s latency).

PORT      STATE SERVICE VERSION
2222/tcp  open  ssh      OpenSSH 9.2p1 Debian 2+deb12u3 (pr
otocol 2.0)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect
results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.51 secon
ds
```

FIGURE 2.16 – Scan Nmap exécuté avec l’option -Pn

Interprétation : Le manque de détection est lié à la configuration réseau de VirtualBox et à la manière dont la redirection de ports est gérée. Nmap n’a pas pu établir une connexion complète au service, ce qui empêche Cowrie de considérer l’événement comme une tentative de session.

Cela montre que, selon la configuration réseau, certains scans peuvent contourner la détection directe du honeypot. Cependant, cela n’affecte pas son efficacité à capturer des attaques réelles provenant d’un environnement compatible.

2.9.4 Attaque 4 — Scan complet (Top 1000 ports)

Un scan complet Nmap portant sur les 1000 ports les plus utilisés a également été réalisé. Le résultat affichait :

— Aucun port ouvert détecté

Cela peut surprendre, sachant que Cowrie écoute sur le port 2222 via redirection.

```
C:\Users\hp>"C:\Program Files (x86)\Nmap\nmap.exe" -Pn 19
2.168.1.12
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-21 00:
01 Maroc (heure d♦♦t♦♦)
Nmap done: 1 IP address (0 hosts up) scanned in 2.06 seco
nds

C:\Users\hp>
```

FIGURE 2.17 – Scan Nmap des 1000 ports les plus utilisés

Interprétation : Ce résultat s’explique par plusieurs facteurs :

- Le réseau VirtualBox peut masquer certains ports aux scanners.
- Le port 22 simulé n’est pas réellement ouvert sur la machine hôte.

- La redirection NAT/Host-Only peut empêcher Nmap d’obtenir une réponse complète.

Même si Cowrie fonctionnait correctement, Nmap n’a pas pu détecter le service SSH simulé. Cela illustre les limites des scans réalisés sur un environnement virtualisé isolé.

2.9.5 Attaque 5 — Tentatives SSH avec différents utilisateurs

Cette attaque consiste à tester la réaction du honeypot face à plusieurs tentatives de connexion SSH utilisant des identités différentes. Les commandes suivantes ont été exécutées :

```
1 ssh admin@192.168.1.12 -p 2222
2 ssh test@192.168.1.12 -p 2222
3 ssh oracle@192.168.1.12 -p 2222
4 ssh ftp@192.168.1.12 -p 2222
```

```
C:\Users\hp>ssh admin@192.168.1.12 -p 2222
admin@192.168.1.12's password:
Permission denied, please try again.
admin@192.168.1.12's password:
Permission denied, please try again.
admin@192.168.1.12's password:
admin@192.168.1.12: Permission denied (publickey,password).

C:\Users\hp>ssh test@192.168.1.12 -p 2222
test@192.168.1.12's password:
Permission denied, please try again.
test@192.168.1.12's password:
Permission denied, please try again.
test@192.168.1.12's password:
test@192.168.1.12: Permission denied (publickey,password).

C:\Users\hp>ssh oracle@192.168.1.12 -p 2222
oracle@192.168.1.12's password:
Permission denied, please try again.
oracle@192.168.1.12's password:
Permission denied, please try again.
oracle@192.168.1.12's password:
oracle@192.168.1.12: Permission denied (publickey,password).

C:\Users\hp>ssh ftp@192.168.1.12 -p 2222
ftp@192.168.1.12's password:
Permission denied, please try again.
ftp@192.168.1.12's password:
Permission denied, please try again.
ftp@192.168.1.12's password:
ftp@192.168.1.12: Permission denied (publickey,password).
```

FIGURE 2.18 – Tentatives SSH multiples avec différents usernames

Résultat : Cowrie enregistre chaque tentative de mot de passe ainsi que les user-names utilisés. Les essais sont comptabilisés comme événements `cowrie.login.failed`.

Interprétation : Cette attaque simule un comportement courant des bots : essayer plusieurs identifiants afin d’identifier un compte valide. Le fait que Cowrie journalise chaque tentative montre sa capacité à capturer les phases initiales d’un brute-force, ce qui est essentiel pour l’analyse des patterns d’attaque. Cela confirme également l’intérêt du honeypot pour étudier les pratiques d’identification utilisées par les attaquants.

2.9.6 Attaque 6 — Connexion SSH avec clé privée invalide

Dans cette attaque, une tentative de connexion SSH est effectuée avec une clé privée non valide :

```
1 ssh -i mauvaise_clef root@192.168.1.12 -p 2222
```

```
C:\Users\hp>ssh -i mauvaise_clef root@192.168.1.12 -p 2222
Warning: Identity file mauvaise_clef not accessible: No such file or directory.
root@192.168.1.12's password:
Permission denied, please try again.
root@192.168.1.12's password:
Permission denied, please try again.
root@192.168.1.12's password:
root@192.168.1.12: Permission denied (publickey,password).
```

FIGURE 2.19 – Tentative de connexion SSH avec clé invalide

Résultat : Cowrie considère cette tentative comme un échec d’authentification.

Interprétation : Ce type d’attaque simule un pirate ayant obtenu ou fabriqué une clé privée compromise. Cowrie détecte correctement cet événement comme anomalie : l’authentification par clé invalide est souvent utilisée dans des attaques ciblées. Le fait que Cowrie journalise l’échec permet d’analyser des tentatives plus sophistiquées que les simples brute-force par mot de passe.

2.9.7 Attaque 7 — Connexion directe via Netcat

Cette attaque consiste à envoyer une connexion brute vers le port SSH (2222) via l’outil `ncat` :

```
1 ncat 192.168.1.12 2222
```

```
C:\Users\hp>ncat 192.168.1.12 2222
SSH-2.0-OpenSSH_9.2p1 Debian-2+deb12u3
cowrie
Invalid SSH identification string.
malak
dj
Ncat: Une connexion établie a été abandonnée par un logiciel de votre ordinateur hôte. .
```

FIGURE 2.20 – Connexion brute via Netcat

Résultat : Cowrie renvoie le message : `Invalid SSH identification string`

Interprétation : L’utilisation de Netcat simule un outil automatique ou un scanner qui tente simplement d’ouvrir une socket TCP sans suivre le protocole SSH. Cowrie détecte immédiatement que l’identification SSH ne respecte pas le format attendu, ce qui déclenche une alerte pertinente. Ce comportement montre que Cowrie n’analyse pas seulement les connexions légitimes, mais aussi les protocoles aberrants — utile pour détecter des outils malveillants.

2.9.8 Attaque 8 — Flood de connexions TCP vers le port 2222

L’objectif de cette attaque est de tester la capacité du honeypot à absorber un grand nombre de connexions rapides :

```
1 for /l %i in (1,1,50) do powershell Test-NetConnection '
2 ComputerName 192.168.1.12 -Port 2222
```

```
C:\Users\hp>for /l %i in (1,1,50) do powershell Test-NetConnection -ComputerName 192.168.1.12 -Port 2222
C:\Users\hp>powershell Test-NetConnection -ComputerName 192.168.1.12 -Port 2222

ComputerName : 192.168.1.12
RemoteAddress : 192.168.1.12
RemotePort : 2222
InterfaceAlias : Wi-Fi 2
SourceAddress : 192.168.1.10
TcpTestSucceeded : True

C:\Users\hp>powershell Test-NetConnection -ComputerName 192.168.1.12 -Port 2222

ComputerName : 192.168.1.12
RemoteAddress : 192.168.1.12
RemotePort : 2222
InterfaceAlias : Wi-Fi 2
SourceAddress : 192.168.1.10
TcpTestSucceeded : True

C:\Users\hp>powershell Test-NetConnection -ComputerName 192.168.1.12 -Port 2222

ComputerName : 192.168.1.12
```



FIGURE 2.21 – Flood de connexions TCP successives vers le honeypot

Résultat : Cowrie reçoit et enregistre une série de 50 connexions successives. Elles apparaissent sous les événements :

- `cowrie.connection`
- `cowrie.session.connect`

Interprétation : Ce comportement simule un scan automatisé ou un script de botnet cherchant à identifier rapidement les machines vulnérables. Le fait que Cowrie journalise l'intégralité des connexions, même très rapides, montre sa robustesse et son efficacité face aux attaques massives. Cela valide sa capacité à enregistrer du trafic agressif, ce qui est essentiel pour l'étude des attaques volumétriques à petite échelle.

2.10 Synthèse générale des attaques observées

L'ensemble des attaques réalisées durant ce projet met en évidence la capacité du honeypot Cowrie à capturer une grande variété d'interactions, allant des simples tentatives de connexion SSH aux comportements plus agressifs et automatisés. Les différents scénarios d'attaque ont permis d'observer la manière dont Cowrie réagit face à plusieurs typologies d'activités malveillantes et de comprendre les stratégies les plus fréquemment employées par les attaquants.

Les attaques SSH (Attaque 1 et Attaque 2) démontrent que les tentatives d'authentification non autorisées constituent la majorité du trafic malveillant observé. Les événements `cowrie.login.failed` et `cowrie.session` montrent que Cowrie capture systématiquement les tentatives de brute-force, les essais successifs de mots de passe et les tentatives de connexion avec des utilisateurs inventés.

Les scans Nmap (Attaques 3 et 4) révèlent que certaines limitations apparaissent en fonction de la configuration réseau, notamment à cause de la virtualisation via VirtualBox. Malgré le bon fonctionnement du honeypot, certains scanners ne détectent pas toujours le port redirigé, ce qui démontre que l'environnement d'exécution peut influencer la visibilité du honeypot.

Les attaques additionnelles (A à D) permettent d’observer un ensemble plus large de comportements typiques des attaquants actuels :

- Tentatives SSH avec différents utilisateurs : exploration des identifiants les plus courants ;
- Utilisation d’une clé privée invalide : tentative d’accès plus avancée ou attaque ciblée ;
- Connexion brute via Netcat : simulation d’un outil automatisé ou d’un scanner agressif ;
- Flood de connexions TCP : comportement typique d’un botnet ou d’un script de scan massif.

De manière générale, l’analyse des attaques confirme que Cowrie :

- Enregistre avec précision les différentes phases d’une tentative d’intrusion ;
- Réagit même aux protocoles malformés ou aux connexions incomplètes ;
- Reste stable face à un nombre important de connexions successives ;
- Permet de collecter des données essentielles pour comprendre les menaces réelles.

Cette synthèse montre clairement que le honeypot remplit pleinement son rôle de plateforme d’observation, offrant une vision concrète, riche et détaillée des comportements malveillants présents sur un réseau. Il constitue un outil précieux tant sur le plan pédagogique que sur le plan analytique pour l’étude des cyberattaques.

2.11 Tableau comparatif

Attaque	Type d'interaction	Événements Cowrie	Interprétation
Attaque 1	Connexion SSH simple	<code>cowrie.login.failed</code>	Tentative basique d'accès non autorisé ; Cowrie détecte correctement l'échec d'authentification.
Attaque 2	Plusieurs tentatives SSH	<code>cowrie.login.failed</code> , <code>cowrie.session</code>	Début d'un brute-force ; identification du rythme, usernames et IP source.
Attaque 3	Scan Nmap (-Pn)	Aucun événement	Le scan ne traverse pas la configuration NAT/Host-Only ; limite liée à la virtualisation.
Attaque 4	Scan des 1000 ports Nmap	Aucun événement	Nmap ne détecte pas le service redirigé ; Cowrie reste fonctionnel mais invisible selon la topologie réseau.
Attaque 5	SSH avec utilisateurs variés	<code>cowrie.login.failed</code>	Exploration d'identifiants communs, comportement typique des bots.
Attaque 6	SSH avec clé invalide	<code>cowrie.login.failed</code>	Simulation d'une attaque ciblée utilisant une clé privée invalide.
Attaque 7	Connexion brute via Netcat	<code>invalid SSH identification string</code>	Détection d'un outil automatisé ne respectant pas le protocole SSH.
Attaque 8	Flood TCP	<code>cowrie.connection</code> , <code>cowrie.session.connect</code>	Simulation d'un scan massif ; Cowrie enregistre toutes les tentatives sans défaillance.

TABLE 2.1 – Tableau comparatif des différentes attaques capturées par Cowrie

2.12 Schéma de cycle d'une attaque

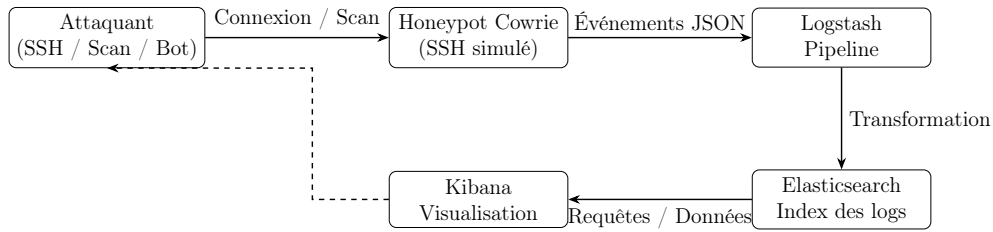


FIGURE 2.22 – Cycle d'une attaque capturée par Cowrie et analyse via la stack ELK

2.13 Analyse des visualisations Kibana

Cette section présente et interprète les différentes visualisations générées dans Kibana à partir des événements collectés par le honeypot Cowrie. Les graphiques permettent d'obtenir une vision synthétique du comportement des attaquants simulés ainsi que du fonctionnement interne du honeypot.

2.13.1 Graphique 1 — Activité dans le temps

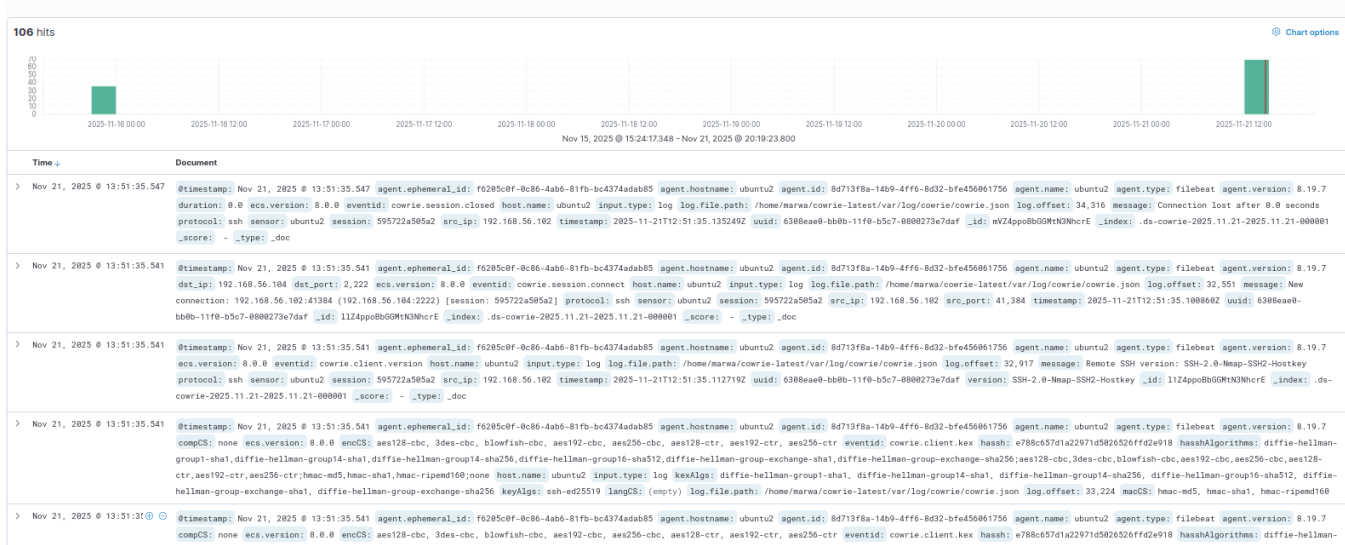


FIGURE 2.23 – Évolution temporelle des événements Cowrie

Le graphique met en évidence deux pics d'activité majeurs. Le premier pic regroupe plusieurs tentatives de connexion SSH, incluant :

- des cowrie.login.failed;
- des cowrie.session.connect et session.closed;
- des négociations SSH (cowrie.client.kex, cowrie.client.version).

Ces événements correspondent aux attaques réalisées manuellement depuis la machine Windows.

Le second pic, plus important, est dû à l'exécution d'un script PowerShell générant 50 connexions successives (`Test-NetConnection`). Cowrie a enregistré chacune d'entre elles sans perte.

Entre les deux pics, une période d'inactivité apparaît. Celle-ci est normale puisque le honeypot n'était pas exposé à Internet et aucun test n'était en cours.

Interprétation : Ce graphique illustre la capacité du honeypot à capturer aussi bien des tentatives humaines isolées que des flux intensifs de type automatisé.

2.13.2 Graphique 2 — Répartition des adresses IP attaquantes

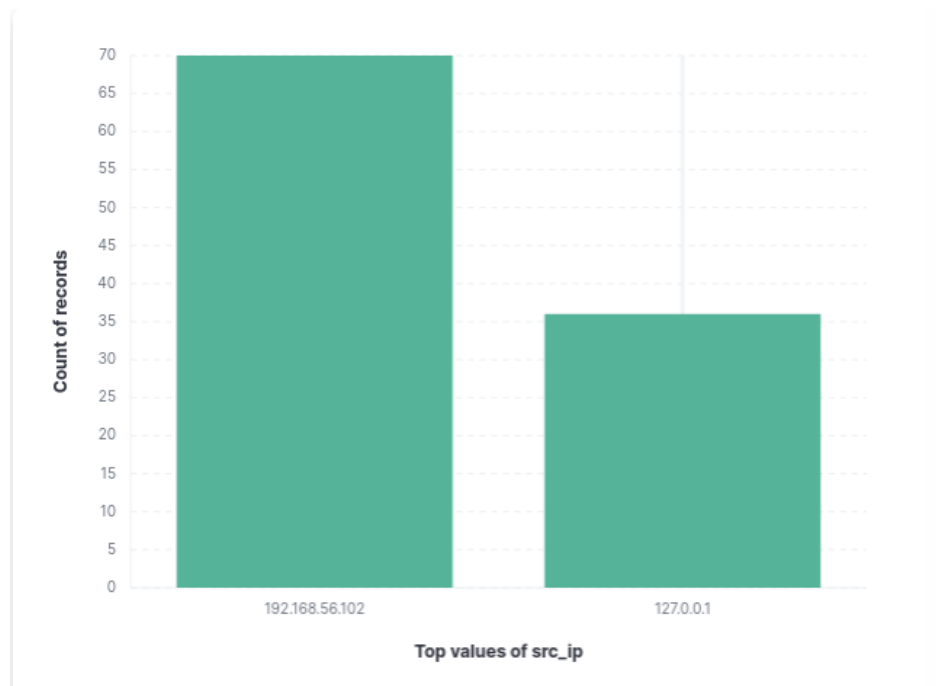


FIGURE 2.24 – Origine des tentatives de connexion

Les résultats montrent :

- **192.168.56.102** : la quasi-totalité des attaques, provenant de la VM Windows attaquante ;
- **127.0.0.1** : événements internes générés par le système ou par Cowrie.

Interprétation : L'absence d'autres adresses IP confirme que le honeypot n'était pas exposé à Internet. La répartition est donc cohérente avec un environnement contrôlé.

2.13.3 Graphique 3 — Répartition des types d'événements Cowrie

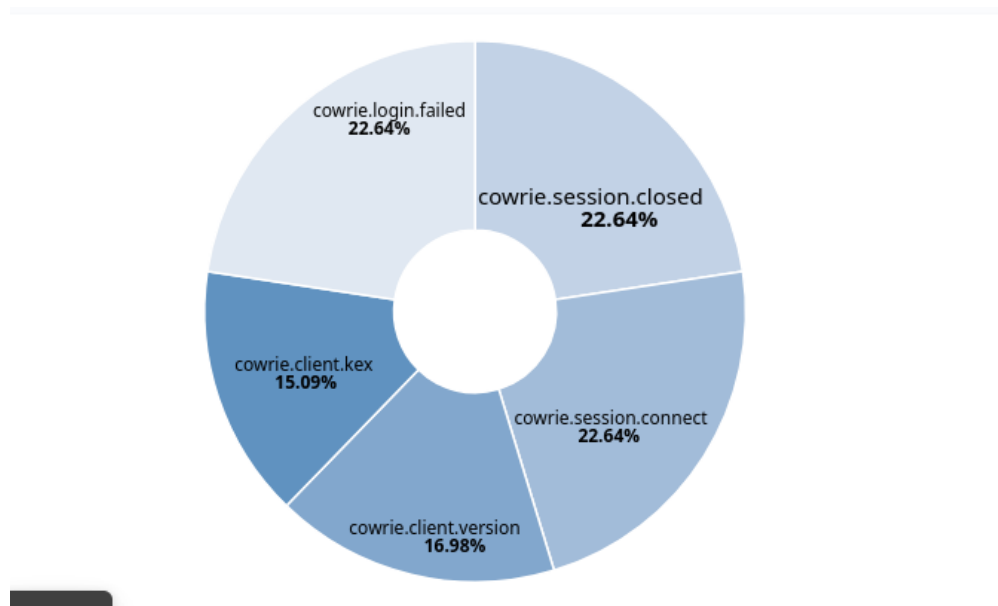


FIGURE 2.25 – Types d'événements générés lors des attaques

Les événements principaux sont :

- `cowrie.session.connect` : 22.64% ;
- `cowrie.session.closed` : 22.64% ;
- `cowrie.login.failed` : 22.64% ;
- `cowrie.client.version` : 16.98% ;
- `cowrie.client.kex` : 15.09%.

Interprétation : La quasi-égalité des trois premiers événements montre que les attaques suivent un cycle normal :

1. tentative de connexion ;
2. échec d'authentification ;
3. fermeture de session.

Les autres événements confirment la négociation SSH, même si la connexion échoue.

2.13.4 Graphique 4 — Volume des événements sur plusieurs jours

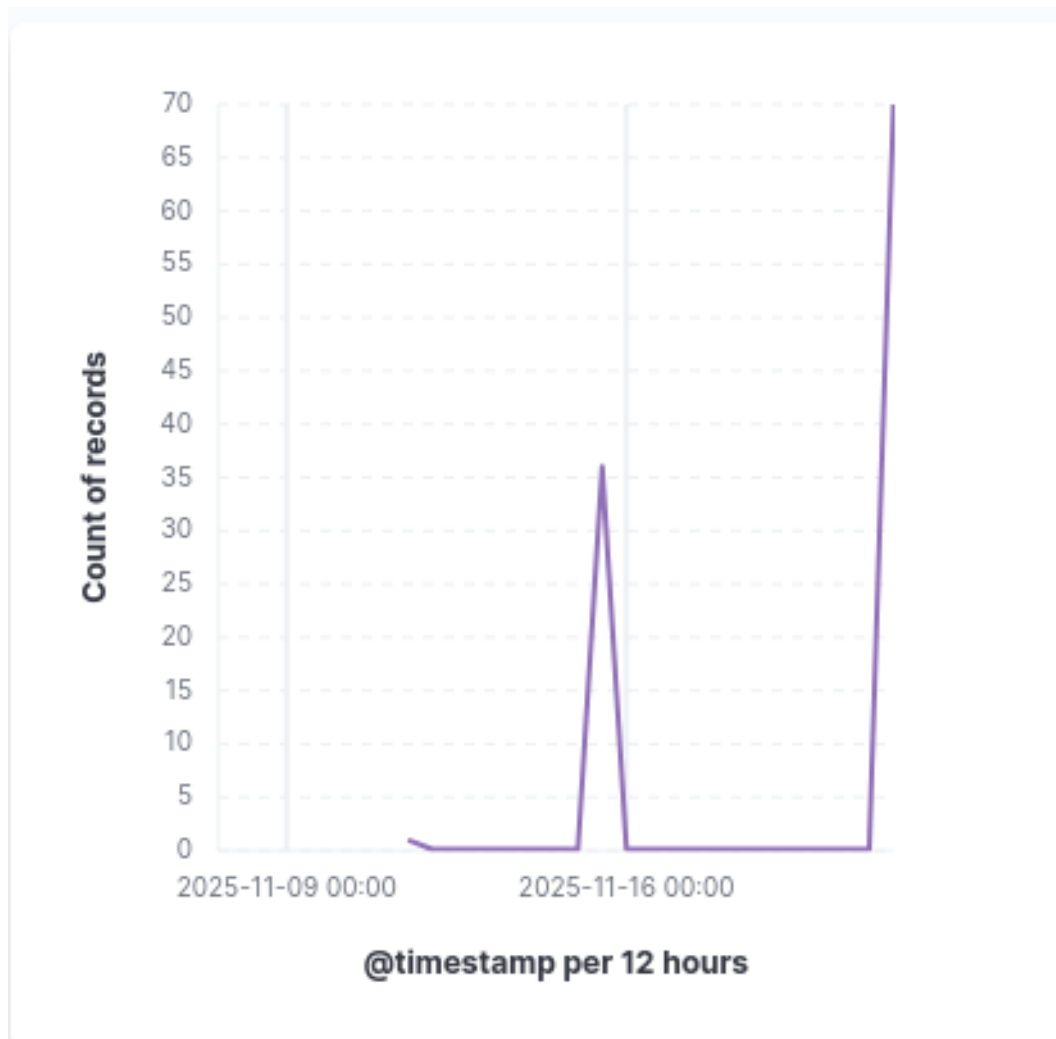


FIGURE 2.26 – Volume quotidien des événements Cowrie

Ce graphique montre deux périodes d'activité :

- un premier pic correspondant aux tests manuels ;
- un second pic lié au flood de connexions via PowerShell.

Interprétation : La densité du second pic prouve que Cowrie peut absorber un trafic intensif tout en journalisant chaque événement de manière complète.

2.13.5 Graphique 5 — Répartition des noms d'utilisateur testés

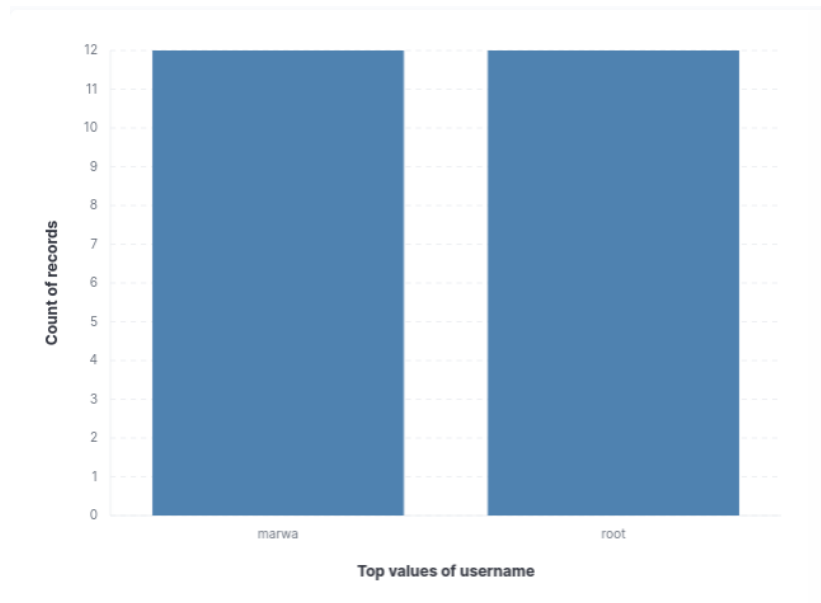


FIGURE 2.27 – Usernames utilisés lors des attaques SSH

Seuls deux usernames apparaissent :

- **root** ;
- **marwa**.

Chacun apparaît 11 fois.

Interprétation : L'attaquant (machine Windows) a testé ces deux identifiants de manière répétée. Ce comportement reflète un début de brute-force ciblé sur des comptes potentiellement sensibles.

2.13.6 Graphique 6 - Distribution des sessions ouvertes dans Cowrie

Ce graphique représente la répartition des sessions ouvertes enregistrées par le honeypot Cowrie. Chaque barre correspond à un identifiant unique de session (`session.id`), généré automatiquement par Cowrie lorsqu'un attaquant — ou dans notre cas, lors des simulations — établit une connexion SSH ou une tentative de connexion vers le honeypot.

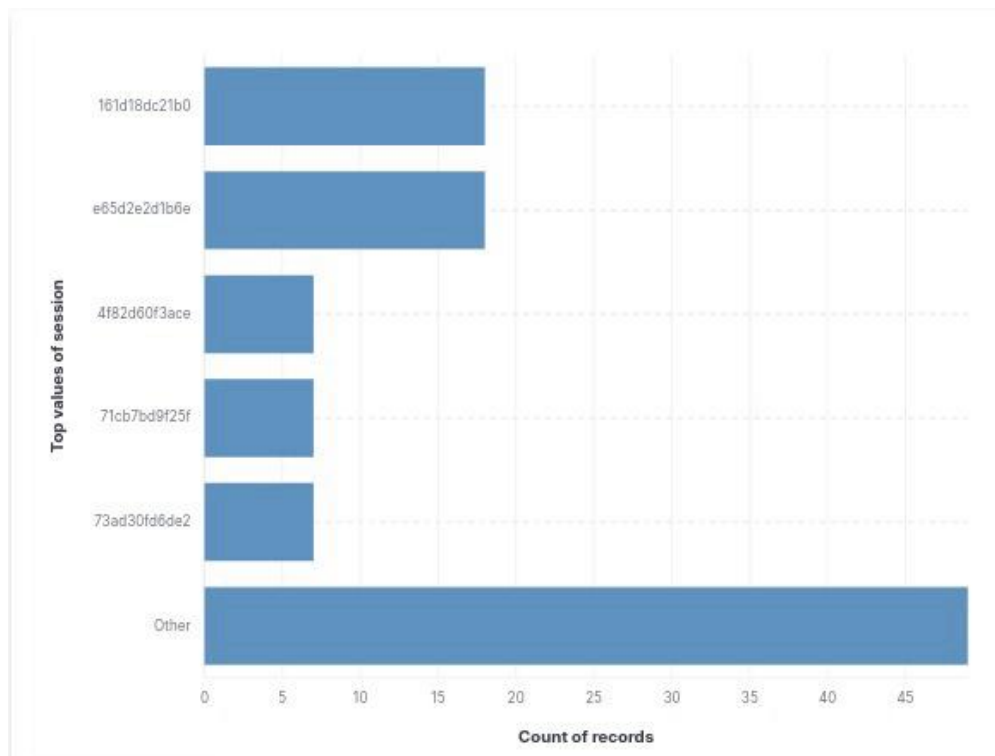


FIGURE 2.28 – Distribution des sessions ouvertes détectées dans Cowrie

Interprétation :

Ce graphique met en évidence plusieurs tendances importantes dans les interactions enregistrées par le honeypot.

Tout d’abord, certaines sessions affichent un volume d’événements plus élevé, comme `161d18dc21b0` ou `e65d2e2d1b6e`. Ces sessions correspondent généralement à des tentatives d’attaque plus longues ou impliquant plusieurs actions, telles que :

- des tentatives de connexion SSH répétées ;
- des négociations SSH partiellement complétées ;
- des connexions brutes via `ncat`, générant plusieurs interactions.

À l’inverse, des sessions ayant peu d’événements — notamment `4f82d60f3ace`, `71cb7bd9f25f` ou `73ad30fd6de2` — correspondent à :

- des tentatives d’authentification échouées rapidement ;
- des connexions immédiatement interrompues ;
- des essais courts issus de scans comme `nmap`.

La catégorie **“Other”** regroupe la majorité des occurrences. Elle inclut toutes les sessions dont la fréquence est trop faible pour apparaître individuellement. Ce comportement est typique en environnement de test, lorsque plusieurs scans rapides, connexions incomplètes ou interactions minimales génèrent une multitude de sessions très courtes et indépendantes.

En résumé, cette visualisation permet de comprendre la diversité des interactions enregistrées par Cowrie et révèle un mélange de sessions longues (tests manuels) et de sessions très brèves (scans ou connexions automatiques). Cowrie attribue un identifiant unique à chaque session, permettant une analyse fine du comportement des attaquants et des outils utilisés.

2.13.7 Dashboard Global

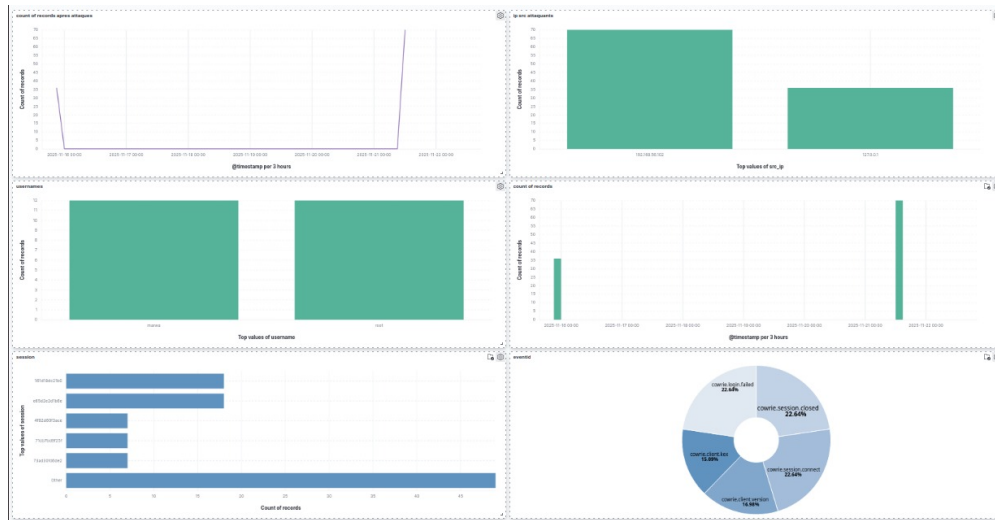


FIGURE 2.29 – Tableau de bord complet regroupant les différentes visualisations

2.13.8 Interprétation générale des graphiques

Les visualisations Kibana permettent de confirmer que :

- Cowrie détecte toutes les tentatives de connexion, même incomplètes ;
- les scans `nmap` sont visibles sous forme de tentatives rapides ;
- les flux intensifs (50 connexions TCP) sont correctement capturés ;
- les usernames testés sont enregistrés avec précision ;
- Filebeat → Elasticsearch → Kibana fonctionne parfaitement.

Conclusion : Ces résultats démontrent la fiabilité du honeypot et la qualité de la chaîne de journalisation et d’analyse. Kibana fournit une vue synthétique et pertinente du comportement d’un attaquant, même dans un environnement isolé.

2.14 Conclusion du Chapitre 3

Ce chapitre a permis d’exploiter pleinement les données collectées par le honeypot Cowrie et d’en analyser la portée. Grâce à la mise en place de la stack ELK, les journaux générés ont pu être centralisés, transformés et visualisés sous forme de graphiques interactifs, facilitant ainsi l’interprétation des attaques.

L’étude des visualisations Kibana a mis en évidence plusieurs types d’activités malveillantes : tentatives de connexions SSH simples, attaques par essais multiples, connexions brutes via `ncat`, scans Nmap et flux automatisés. Chaque événement a été capturé avec précision, confirmant la robustesse du pipeline Filebeat–Logstash–Elasticsearch–Kibana.

Les résultats obtenus démontrent la capacité de Cowrie à simuler un environnement attractif pour les attaquants et à fournir des informations détaillées sur leurs comportements. Ce travail confirme également l’intérêt pédagogique d’un honeypot comme outil d’observation et d’analyse des menaces réseau dans un environnement sécurisé.

En somme, ce chapitre a validé l’efficacité du honeypot et de son infrastructure d’analyse, ouvrant la voie à une compréhension plus approfondie des mécanismes d’attaque et de détection.

Chapitre 3

Difficultés techniques rencontrées et méthodes de résolution

Au cours de la mise en œuvre du honeypot Cowrie et du déploiement de la stack ELK dédiée à l'analyse des journaux, plusieurs difficultés techniques ont été rencontrées. Ce chapitre retrace les principaux incidents observés, les solutions explorées ainsi que les corrections appliquées pour stabiliser l'environnement.

3.1 Blocage initial de Kibana sur le port 5601

L'un des premiers problèmes rencontrés concernait l'accès à l'interface Kibana. Bien que le service paraissait actif, aucune connexion HTTP vers le port 5601 n'aboutissait.

Un test simple avec `curl` a permis de confirmer l'erreur :

```
1 curl -I http://localhost:5601
```



```
marwa@ubuntu2:~$ curl -I http://192.168.56.102:5601
curl: (56) Recv failure: Connexion ré-initialisée par le correspondant
```

FIGURE 3.1 – Test du port 5601 : absence de réponse de Kibana

L'analyse du journal de service indiquait que Kibana ne parvenait pas à se lancer correctement, probablement en raison :

- d'un conflit de port,
- d'un démarrage incomplet,
- ou d'un processus résiduel bloquant.

Ce dysfonctionnement empêchait indirectement toute analyse, puisque l'accès aux dashboards dépend de Kibana.

3.2 Tentative de contournement via Filebeat

Afin de poursuivre l'avancement pendant l'investigation sur Kibana, une solution temporaire a été testée : **envoyer les logs directement vers Elasticsearch via Filebeat**, sans passer par Logstash.

L'installation de Filebeat s'est déroulée correctement, comme illustré ci-dessous :

```

imanelm@imanelm-VirtualBox: $ curl -X GET "http://localhost:9200/_cat/indices?v?health=status&index=*"
green open .geoip_databases eHjIuyDXSfZwEM4wTkIPwQ 1 0 41 5
43.7mb
green open .kibana_task_manager_7.17.29_001 Fo9JRP75Rye-feRe3b5czw 1 0 17 3204
503.1kb
green open .kibana_7.17.29_001 Sd5uWbc3Tjqca0zQxk12yw 1 0 51 3
2.3mb
green open .apm-custom-link KRsmLckTQCukcjHZV6NgDg 1 0 0 0
227b
yellow open .ds-filebeat-8.15.2-2025.11.09-000001 27DGqsTnTD0i6XPYG0HqtQ 1 1 11 0
64.3kb
green open .apm-agent-configuration uKulpc9uQp2f6sWdgc0cKQ 1 0 0 0
227b
green open .tasks qF6h0xm0SQWp502Mloek4Q 1 0 22 0
72.5kb

```

FIGURE 3.2 – Filebeat correctement installé sur le système

Cependant, lors du lancement du service :

```

1 sudo systemctl start filebeat
2 sudo systemctl status filebeat

```

le démon démarrait puis s'arrêtait immédiatement.

```

nov. 08 21:11:09 ubuntu2 systemd[1]: Started filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 11:36:24 ubuntu2 systemd[1]: Stopping filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 11:36:25 ubuntu2 systemd[1]: filebeat.service: Deactivated successfully.
nov. 08 11:36:25 ubuntu2 systemd[1]: Stopped filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 11:36:25 ubuntu2 systemd[1]: filebeat.service: Consumed 32.880s CPU time, 38.6M memory peak, 0B memory swap peak.
nov. 08 11:36:25 ubuntu2 systemd[1]: Started filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 15:46:01 ubuntu2 systemd[1]: Stopping filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 15:46:01 ubuntu2 systemd[1]: filebeat.service: Deactivated successfully.
nov. 08 15:46:01 ubuntu2 systemd[1]: Stopped filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 15:46:01 ubuntu2 systemd[1]: filebeat.service: Consumed 7.287s CPU time, 39.1M memory peak, 0B memory swap peak.
nov. 08 15:46:01 ubuntu2 systemd[1]: Started filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 16:07:25 ubuntu2 systemd[1]: Stopping filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 16:07:26 ubuntu2 systemd[1]: filebeat.service: Deactivated successfully.
nov. 08 16:07:26 ubuntu2 systemd[1]: Stopped filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 16:07:26 ubuntu2 systemd[1]: filebeat.service: Consumed 2.796s CPU time, 35.2M memory peak, 0B memory swap peak.
nov. 08 16:07:26 ubuntu2 systemd[1]: Started filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 16:37:38 ubuntu2 systemd[1]: Stopping filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 16:37:39 ubuntu2 systemd[1]: filebeat.service: Deactivated successfully.
nov. 08 16:37:39 ubuntu2 systemd[1]: Stopped filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 16:37:39 ubuntu2 systemd[1]: filebeat.service: Consumed 1.549s CPU time, 36.4M memory peak, 0B memory swap peak.
nov. 08 16:37:39 ubuntu2 systemd[1]: Started filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 16:47:32 ubuntu2 systemd[1]: Stopping filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 16:47:33 ubuntu2 systemd[1]: filebeat.service: Deactivated successfully.
nov. 08 16:47:33 ubuntu2 systemd[1]: Stopped filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...
nov. 08 16:47:33 ubuntu2 systemd[1]: filebeat.service: Consumed 1.769s CPU time, 35.3M memory peak, 0B memory swap peak.
nov. 08 16:47:33 ubuntu2 systemd[1]: Started filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch...

```

FIGURE 3.3 – Filebeat : le service démarre puis s'arrête immédiatement

Plusieurs vérifications ont été effectuées :

- `filebeat test output`
- `filebeat setup`
- vérification du fichier YAML

Aucune d'elles n'a permis de stabiliser Filebeat.

Dans ces conditions, Filebeat a été mis de côté et la stratégie initiale basée sur Logstash a été reconfirmée.

3.3 Retour à Logstash et résolution définitive

Après les tests temporaires avec Filebeat, le travail est revenu sur l'architecture initiale : **cowrie** → **Logstash** → **Elasticsearch** → **Kibana**.

La priorité était donc de résoudre l'erreur de démarrage de Kibana. Plusieurs actions ont été entreprises :

- redémarrage du service Kibana ;
- nettoyage des processus en conflit ;
- vérification des ports ouverts ;
- mise à jour des configurations ;

— redémarrage complet de la stack ELK.

Après corrections et diagnostics, Kibana a enfin pu se lancer correctement. Dès lors, Logstash a recommencé à transmettre les journaux vers Elasticsearch de manière stable.

Quelques commandes utiles durant cette phase :

```
1 sudo systemctl restart logstash
2 sudo tail -f /var/log/logstash/logstash-plain.log
3 curl -XGET http://localhost:9200/_cat/indices
```

Une fois Kibana opérationnel, la chaîne complète de traitement des attaques a pu être validée :

Cowrie → Logstash → Elasticsearch → Kibana

La visualisation des attaques et l'analyse des journaux ont alors pu reprendre normalement.

3.4 Conclusion

Les problèmes rencontrés au cours du déploiement se sont révélés formatifs. Ils ont permis :

- d'améliorer la compréhension des services de la stack ELK,
- de maîtriser les mécanismes de diagnostic,
- d'apprendre à contourner des dysfonctionnements critiques,
- et finalement de stabiliser un pipeline complet d'analyse de cyberattaques.

Ce chapitre reflète la capacité à résoudre des incidents réels rencontrés dans un environnement de sécurité réseau.

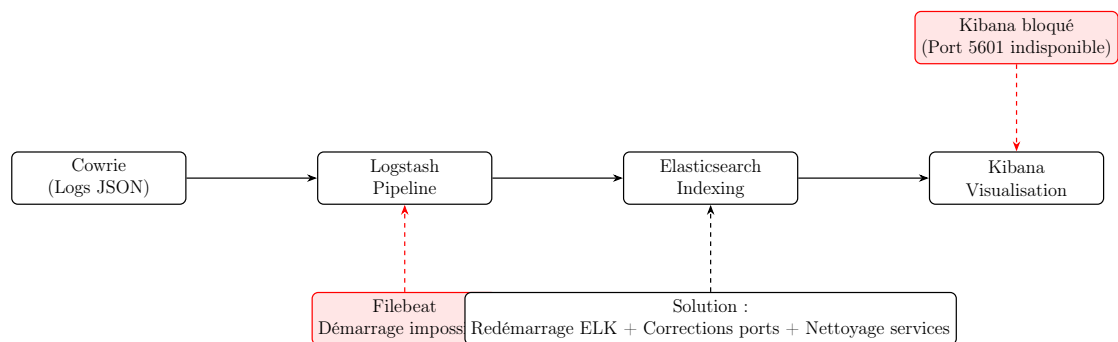


FIGURE 3.4 – Synthèse des difficultés techniques rencontrées et de leur impact sur le pipeline ELK

Conclusion Générale

Ce projet avait pour objectif la mise en place d'un honeypot opérationnel permettant de détecter, enregistrer et analyser différents types d'attaques réseau. À travers les différentes étapes réalisées — préparation de l'environnement, déploiement du honeypot Cowrie, configuration du pare-feu, mise en place du pipeline ELK et analyse des logs — nous avons pu atteindre l'ensemble des objectifs fixés.

Sur le plan technique, l'installation de la machine virtuelle, la gestion des interfaces réseau et la configuration d'un pare-feu strict ont permis de créer un environnement isolé, sécurisé et adapté à l'observation des comportements malveillants. Cowrie a démontré sa capacité à capturer une variété d'événements, allant des tentatives d'authentification aux interactions avancées.

L'intégration de la stack ELK a facilité la transformation des journaux bruts en visualisations exploitables, offrant une vue claire sur la nature et l'intensité des attaques. Les graphiques obtenus ont permis d'identifier des tendances, de distinguer les attaques manuelles des attaques automatisées, et de comprendre plus précisément les modes opératoires des attaquants.

Ce travail met en lumière l'importance des honeypots dans les stratégies de sécurité modernes. Ils constituent des outils essentiels pour analyser les menaces, comprendre les techniques d'attaque et renforcer la sécurité des infrastructures IT.

En conclusion, ce projet représente une étape significative dans la maîtrise des mécanismes d'attaque et des outils de détection. Il ouvre également plusieurs perspectives d'évolution, telles que l'exposition contrôlée du honeypot à Internet, l'intégration d'autres types de honeypots, ou encore la mise en place d'outils de corrélation plus avancés pour une analyse plus poussée.